

Terms used in microprocessor literature

Bit : A digit of the binary number or code is called bit.

Nibble : The 4-bit (4-digit) binary number or code is called nibble.

Byte : The 8-bit (8-digit) binary number or code is called byte.

Word : The 16-bit (16-digit) binary number or code is called word.

Double Word : The 32-bit (32-digit) binary number or code is called double word.

Multiple Word : The 64, 128, ... bit /digit binary numbers or codes are called multiple words.

Data : The quantity (binary number/code) operated by an instruction of a program is called data. The size of data is specified as bit, byte, word, etc.

Address : Address is an identification number in binary for memory locations.

The 8086 processor uses a 20-bit address for memory.

Memory Word Size : The memory word size or addressability is the size of binary information (or Addressability) that can be stored in a memory location. The memory word size for an 8086 processor-based system is 8-bit.

Microprocessor: The microprocessor is a program controlled semiconductor device(IC), which fetches (from memory), decodes and executes instructions. It is used as CPU (Central Processing Unit) in computers. The basic functional blocks of a microprocessor are ALU (Arithmetic

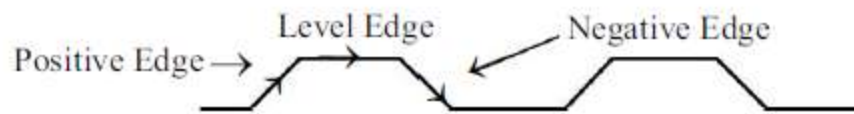
Logic Unit), an array of registers and a control unit. The microprocessor is identified with the size of data, the ALU of the processor can work with at a time. The 8086 processor has a 16-bit ALU, hence it is called a 16-bit processor. The 80486 processor has a 32-bit ALU, hence it is called a 32-bit processor.

Bus : A bus is a group of conducting lines that carries data, address and control signals. Buses can be classified into Data bus, Address bus and Control bus. The group of conducting lines that carries data is called data bus. The group of conducting lines that carries address is called address bus. The group of conducting lines that carries control signals is called control bus.

CPU Bus : The group of conducting lines that are directly connected to the microprocessor is called CPU bus. In a CPU bus, the signals are multiplexed, i.e., more than one signal is passed through the same line but at different timings.

System Bus : The group of conducting lines that carries data, address and control signals in a microcomputer system is called System bus. Multiplexing is not allowed in a system bus

Clock : A clock is a square wave used to synchronize various devices in the microprocessor and in the system. Every microprocessor system requires a clock for its functioning. The time taken for the microprocessor and the system to execute an instruction or program are measured only in terms of the time period of its clock.



A clock has three edges : rising edge (positive edge), level edge and falling edge (negative edge). The device is made sensitive to any one of the edges for better functioning (it means that the device will recognize the clock only when the edge is asserted or arrived).

Tristate Logic : Almost all the devices used in a microprocessor-based system use tristate logic. In devices with tristate logic, three logic levels will be available : **High state**, **Low state** and **High impedance state**.

The high and low level states are normal logic levels for data, address or control signals.

The high impedance state is an electrical open-circuit condition. The high impedance state is provided to keep the device electrically isolated from the system.

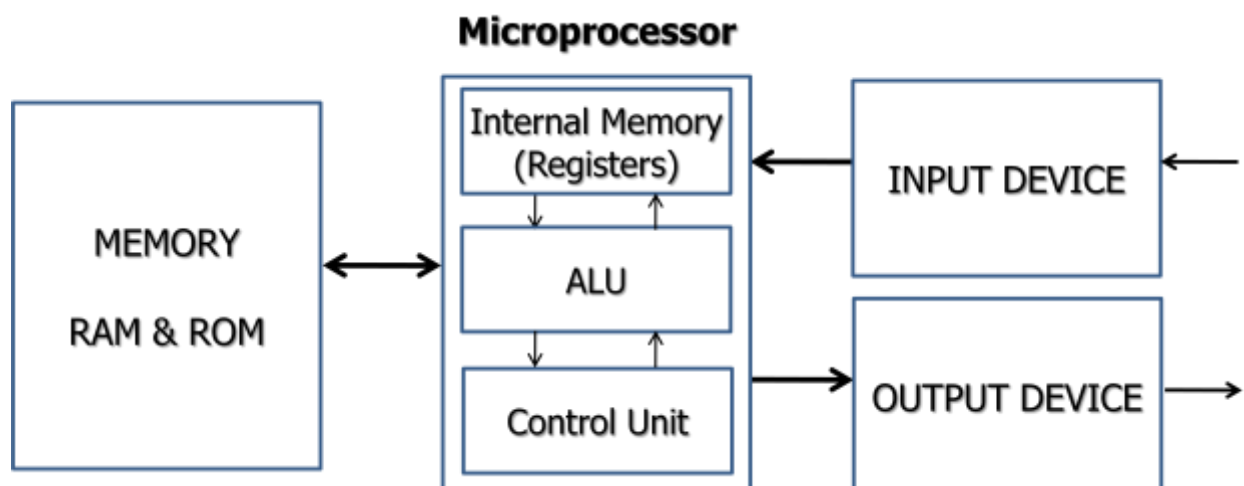
The **tristate devices** will normally remain in high impedance state and their pins are physically connected in the system bus but electrically isolated. In high impedance state, they cannot receive or send any signal or information. These devices are provided with chip enable/chip select pins. When the signal at this pin is asserted to the right level, they come out from high impedance state to normal levels.

Microcomputer: The term microcomputer is generally synonymous with personal computer, or a computer that depends on a microprocessor.

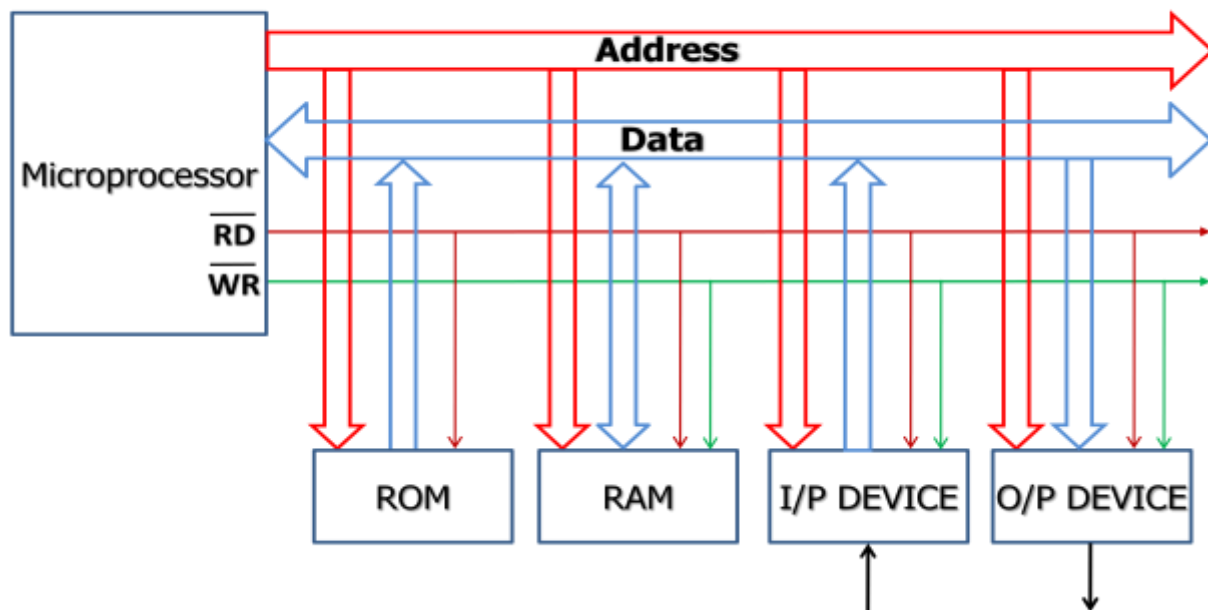
- Microcomputers are designed to be used by individuals, whether in the form of PCs, workstations or notebook computers.
- A microcomputer contains a CPU on a microchip (the microprocessor), a memory system (typically ROM and RAM), a bus system and I/O ports, typically housed in a motherboard.

Microprocessor:

- A microprocessor is a clock-driven semiconductor device consisting of electronic logic circuits manufactured by using either a large-scale integration (LSI) or very-large-scale integration (VLSI) technique.
- The microprocessor is capable of performing various computing functions and making decisions to change the sequence of program execution.
- In large computers, a CPU performs these computing functions. The Microprocessor resembles a CPU exactly. The microprocessor is in many ways similar to the CPU, but includes all the logic circuitry including the control unit, on one chip.
- The microprocessor can be divided into three segments for the sake of clarity. – They are: arithmetic/logic unit (ALU), register array, and control unit. A comparison between a microprocessor, and a computer is shown below:



Simplified MPU



- **Arithmetic/Logic Unit:** This is the area of the microprocessor where various computing functions are performed on data. The ALU unit performs such arithmetic operations as addition and subtraction, and such logic operations as AND, OR, and exclusive OR.
- **Register Array:** This area of the microprocessor consists of various registers identified by letters such as B, C, D, E, H, and L. These registers are primarily used to store data temporarily during the execution of a program and are accessible to the user through instructions.
- **Control Unit:** The control unit provides the necessary timing and control signals to all the operations in the microcomputer. It controls the flow of data between the microprocessor and memory and peripherals.
- **Memory:** Memory stores such binary information as instructions and data, and provides that information to the microprocessor whenever necessary. To execute programs, the microprocessor reads instructions and data from memory and performs the computing operations in its ALU section. Results are either transferred to the output section for display or stored in memory for later use. Read-Only memory (ROM) and Read/Write memory (R/W), popularly known as Random- Access memory (RAM).

- **I/O (Input/Output):** It communicates with the outside world. I/O includes two types of devices: input and output; these I/O devices are also known as peripherals.
- **System Bus:** The system bus is a communication path between the microprocessor and peripherals: it is nothing but a group of wires to carry bits.

Evolution of Microprocessors

The first μ P was introduced in 1971 by Intel Corporation. This was the Intel 4004, a processor on a single chip. It had the capability of performing simple arithmetic and logical operations.

The first 8 bit μ P, which would perform arithmetic and logic operations on 8 bit words, was introduced in 1973, by Intel. This was 8008 that was followed by an improved version- the 8080 from the same company.

The μ Ps introduced between 1971 and 1972 were the **first generation** systems. They were designed using the PMOS technology. This technology provided low cost, slow speed and low output currents and was compatible with TTL.

After 1973, the **second generation** μ Ps such as Motorola 6800 and 6809, Intel 8085 and Zilog Z80 evolved. These μ Ps were fabricated using NMOS technology. The distinction between the 1st & 2nd generation devices was primarily the use of new a semiconductor technology to fabricate the chips.

After 1978, the **3rd generation** microprocessors were introduced. Typical μ Ps were Intel 8086/ 80186/ 80286 and Motorola 68000/ 68010. These μ Ps were designed using HMOS technology. HMOS provides the following advantages over NMOS.

The third generation introduced in 1978 was typically separated by the Intel 8086 iAPX8086, iAPX80186, iAPX80286, Zilog 8000, and the Motorola 68000 which are 16-bit μ Ps with minicomputer like performance.

In 1980, the fourth generation μ Ps were evolved. Intel introduced the first commercial 32 bit microprocessor, Intel 432. Since 1985, more 32-bit μ Ps have

been introduced. These include Intel iAPX80386, Intel 80486, Motorola MC68020/68030/68040, National semiconductor NS 32032.

These processors were fabricated using the low power version of HMOS technology called HCMOS, and they include an on-chip RAM called the cache memory to speed up program execution.

Processor	Year of launch	No. of transistor	Initial clock	Data bus (bit)	Address Bus (bit)	Memory addressing Capacity
4004	1971	2300	108 KHz	4-bit	12-bit	4 K
8008	1972	3500	200KHz	8-bit	14-bit	16K
8080	1974	6000	2MHz	8-bit	16 -bit	64 K
8085	1976	6500	5 MHz	8-bit	16 -bit	64 K
8086	1978	29000	5-10MHz	16-bit	20-bit	1M
80186	1982	55000	6 MHz	16-bit	20-bit	1MB
80286	1982	1,34000	8 MHz	16-bit	24-bit	16M
80386	1985	2,75000	16 MHz	32-bit	32-bit	4G
80486	1989	1.2 million(M)	25 MHz	32-bit	32-bit	4G
Pentium	1993	3.1M	60MHz	32/64	32-bit	4G
Pentium Pro	1995	5.5M	150MHz	32/64	36-bit	64G
Pentium II	1997	8.8M	233MHz	64-bit	36-bit	64G
Pentium III	1999	9.5M	650MHz	64-bit	36-bit	64G
Pentium IV	2000	42M	1.4GHz	64-bit	36-bit	64G

Dual core	2006	291M	1.2GHz-3GHz	64-bit	32-bit	4G
Core i7	2008	781M	2.66GHz	64-bit	32-bit	4G
Core i5	2009	781M	3.6GHz	64-bit	32-bit	4G
Core i3	2010	781M	3.3GHz	64-bit	32-bit	4G

Microcomputer

Microprocessor is a central processing unit (CPU) with its related timing functions on a single chip. A microprocessor combined with memory and input/output devices forms a microcomputer. Therefore, the basic components of a microcomputer are:

- ❖ CPU
- ❖ Program memory
- ❖ Data memory
- ❖ Output ports
- ❖ Input ports
- ❖ Clock generator.

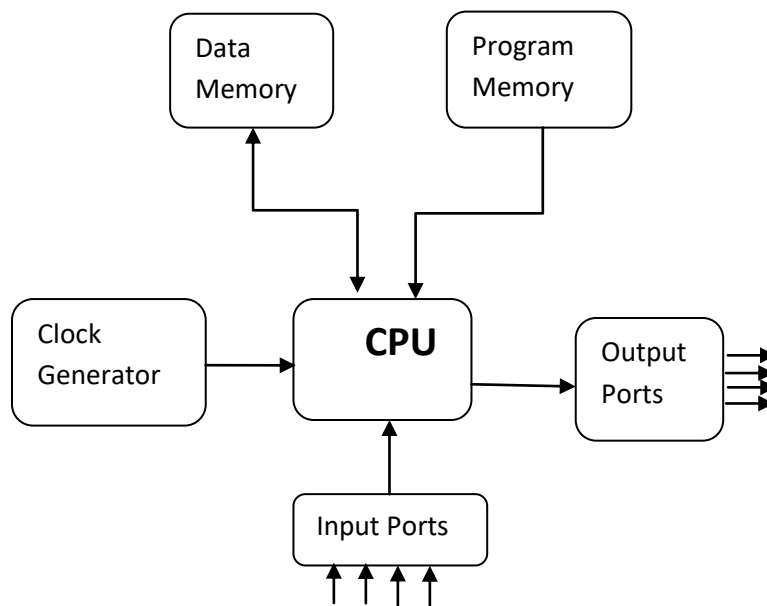


Figure: Basic Components of Microcomputer

Central Processing Unit: The CPU consists of ALU (Arithmetic and Logic Unit), register unit and control unit. The CPU fetches the stored instructions from the program memory, data word from data memory or from an input device and after processing the data stores the result in data memory or sends it to an output device.

Program Memory: The basic task of a microcomputer system is to ensure that its CPU executes the desired instructions sequence i.e., the program properly. The instructions sequence is stored in the program memory. On initialization- usually on power up or manual reset the processor starts executing the instructions from a predetermined location in program memory.

Data Memory: A microcomputer manipulates data according to the algorithm given by the instruction in the program in the program memory. These instructions may require intermediate results to be stored. The functional block in the microcomputer used for this storage is the data memory. Microprocessors also have a small amount of memory in the form of internal registers which can also be used if available for such storage. External data memory is needed if the storage requirement is more.

Input/Output Ports: The input & output ports provide the microcomputer the capability to communicate with the outside world.

The input ports allow data to pass from the outside world to the μ c data which will be used in the data manipulation being done by the microcomputer to send data to output devices.

The user can enter instruction (i.e. program) and data in memory through input devices such as keyboard, or simple switches, CRT, disk devices, tape or card readers

Clock Generator: Operations inside the microprocessor as well as in other parts of the microcomputer are usually synchronous by nature. This is done so that events in different parts of the system can proceed in a systematic fashion.

The clock needed to perform this synchronous operation is provided by the clock generator. The clock generator generates the appropriate clock periods during which instruction executions are carried out by the microprocessor.

8085 Microprocessor

The salient features of 8085 microprocessor are:

- It is an 8 bit microprocessor.
- It is manufactured with N-MOS technology.
- It has 16-bit address bus and hence can address up to $2^{16} = 65536$ (64K) memory locations through $A_0 - A_{15}$.
- The lower 8 lines of address bus and 8 lines of data bus are multiplexed $AD_7 - AD_0$.
- Data bus is a group of 8 lines D_0 to D_7
- It supports external interrupt request.
- A Program counter (PC) of 16-bit
- A stack pointer (SP) of 16-bit
- Six 8-bit general purpose register arranged in pairs: BC, DE, HL.
- It requires a signal +5V power supply and operates at 3MHZ single phase clock.
- It is enclosed with 40 pins DIP (Dual in line package).

Architecture of INTEL 8085/8085A

- Intel 8085A is one of the most popular 8-bit microprocessors capable of addressing 64KB of memory and its architecture is simple. The architecture of 8085 includes the ALU, timing and control unit, instruction register and decoder, register array, interrupt control and serial I/O control in a package of 40 pins, requires +5V single power supply and can operate with a 3MHz single phase clock.

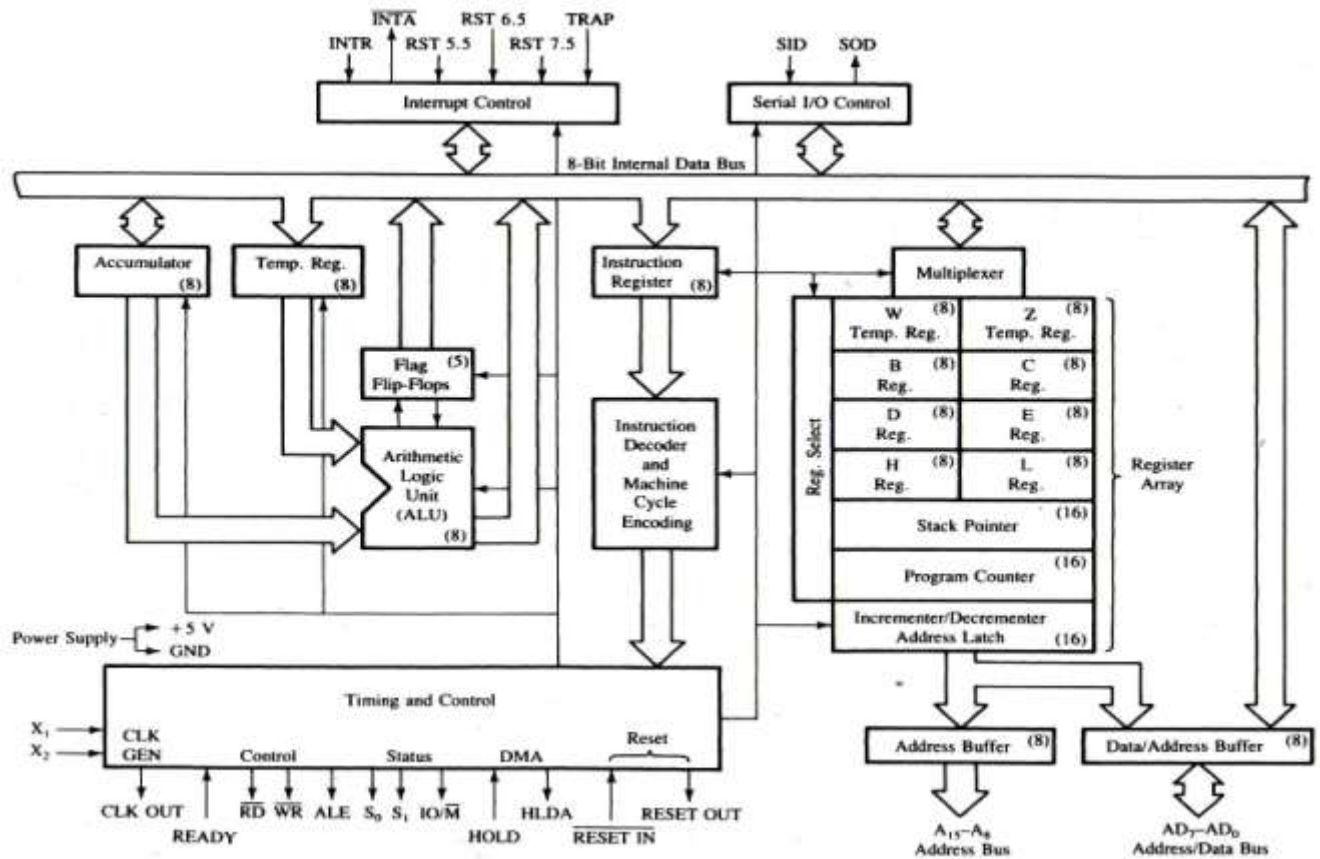


Figure: Internal Architecture of 8085/8085A

Arithmetic Logic Unit

The ALU performs the actual numerical and logic operation such as 'add', 'subtract', 'AND', 'OR', etc. Uses data from memory and from Accumulator to perform arithmetic. Always stores result of operation in Accumulator.

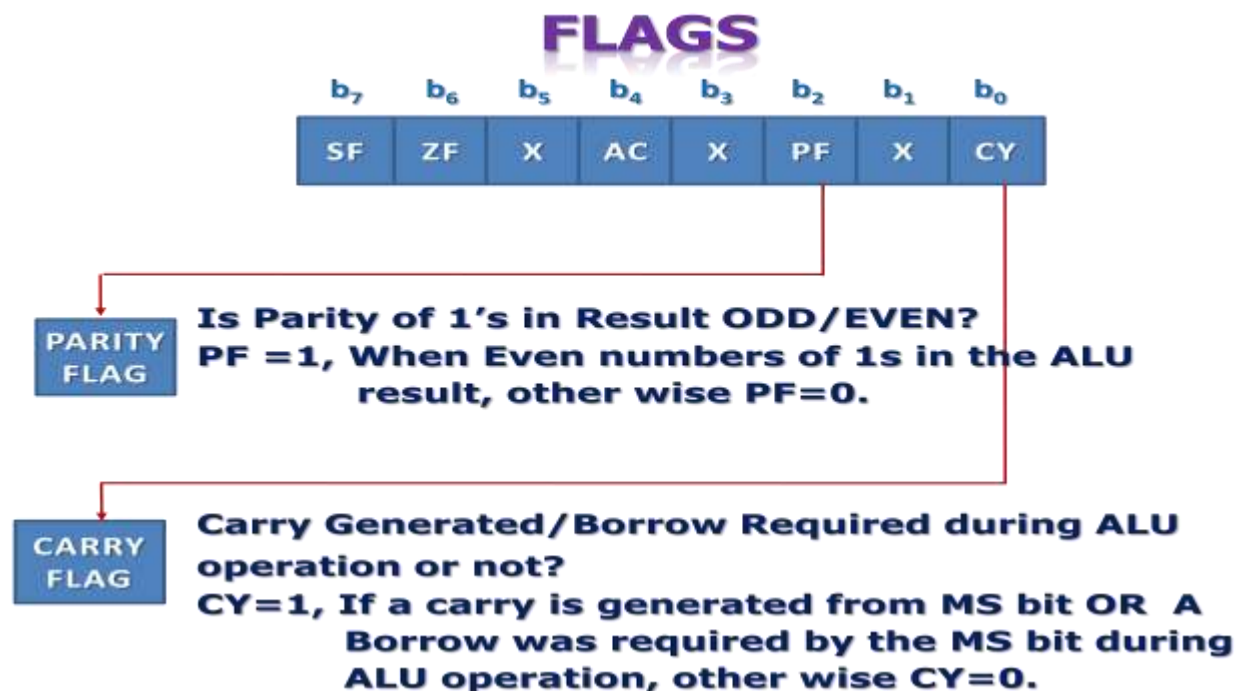
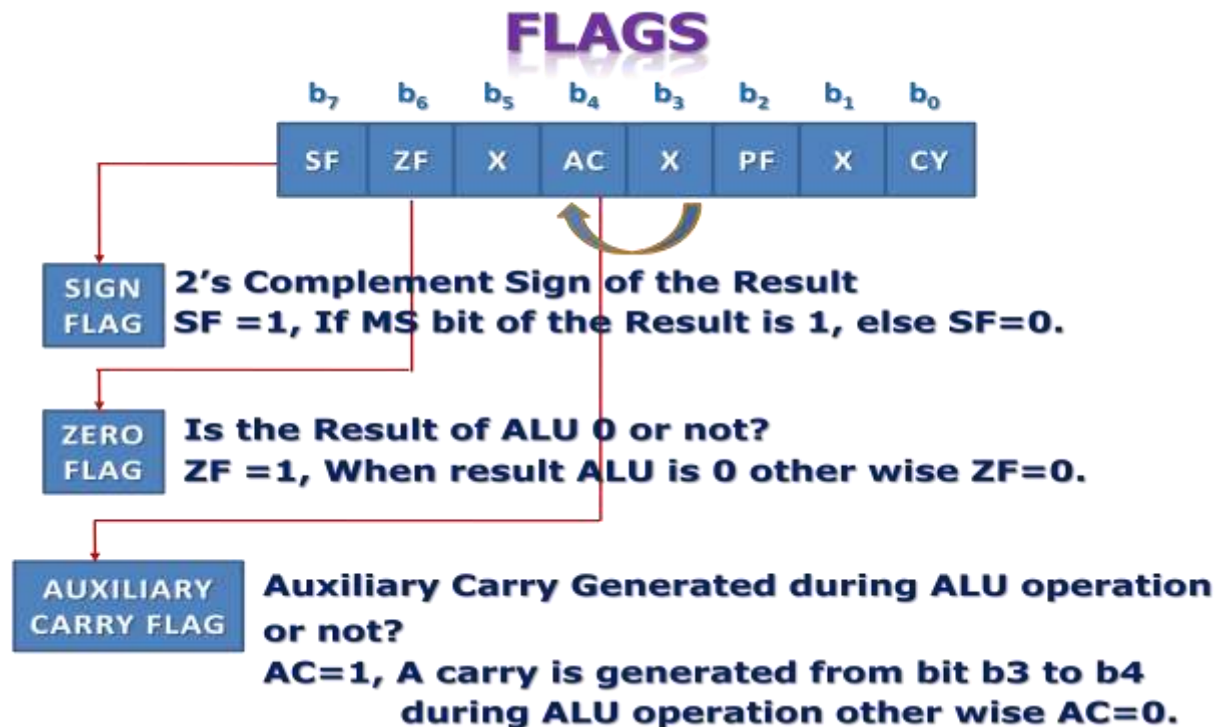
Accumulator

The accumulator is an 8-bit register that is a part of arithmetic/logic unit (ALU). This register is used to store 8-bit data and to perform arithmetic and logical operations. The result of an operation is stored in the accumulator. The accumulator is also identified as register A.

Flags

The ALU includes five flip-flops, which are set or reset after an operation according to data conditions of the result in the accumulator and other registers. They are called Zero(Z), Carry (CY), Sign (S), Parity (P), and Auxiliary Carry (AC) flags; they are listed in the Table and their bit positions in the flag register are

shown in the Figure below. The most commonly used flags are Zero, Carry, and Sign. The microprocessor uses these flags to test data conditions.



FLAGS

ADD 23H and 15H

23 = 0 0 1 0 0 0 1 1
15 = 0 0 0 1 0 1 0 1

38H 0 0 1 1 1 0 0 0

No CARRY from MS
bit, **CY = 0**.

MS bit of the SUM is 0,
SF = 0.

Result is NON- ZERO ,
ZF = 0.

There are 3 1s, **PF = 0**.

There is NO CARRY from
bit b_3 to bit b_4 , **AC = 0**.

FLAGS (CONTD.)

ADD 69H and 18H

69 = 0 1 1 0 1 0 0 1
18 = 0 0 0 1 1 0 0 0

81H 1 0 0 0 0 0 0 1

No CARRY from MS
bit, **CY = 0**.

MS bit of the SUM is 1,
SF = 1.

Result is NON- ZERO ,
ZF = 0.

There are 2 1s, **PF = 1**.

There is a CARRY from bit
 b_3 to bit b_4 , **AC = 1**.

Why result of SUM appears Negative (i.e. SF=1)

FLAGS

ADD CB and E9

CB = 1 1 0 0 1 0 1 1

E9 = 1 1 1 0 1 0 0 1

1 1 0 1 1 0 1 0 0

There is a CARRY from MS bit, CY = 1.

MS bit of the SUM is 1, SF = 1.

Result is NON-ZERO, ZF = 0.

There are 4 1s, PF = 1.

There is a CARRY from bit b_3 to bit b_4 , AC = 1.

These flags have critical importance in the decision-making process of the microprocessor. The conditions (set or reset) of the flags are tested through the software instructions. For example, the instruction JC (Jump on Carry) is implemented to change the sequence of a program when CY flag is set. The thorough understanding of flag is essential in writing assembly language programs.

Control Unit

Generates signals within microprocessor to carry out the instruction, which has been decoded. In reality causes certain connections between blocks of the uP to be opened or closed, so that data goes where it is required, and so that ALU operations occur.

Registers

The 8085/8080A-programming model includes six registers, one accumulator, and one flag register, as shown in Figure. In addition, it has two 16-bit registers: the stack pointer and the program counter. They are described briefly as follows.

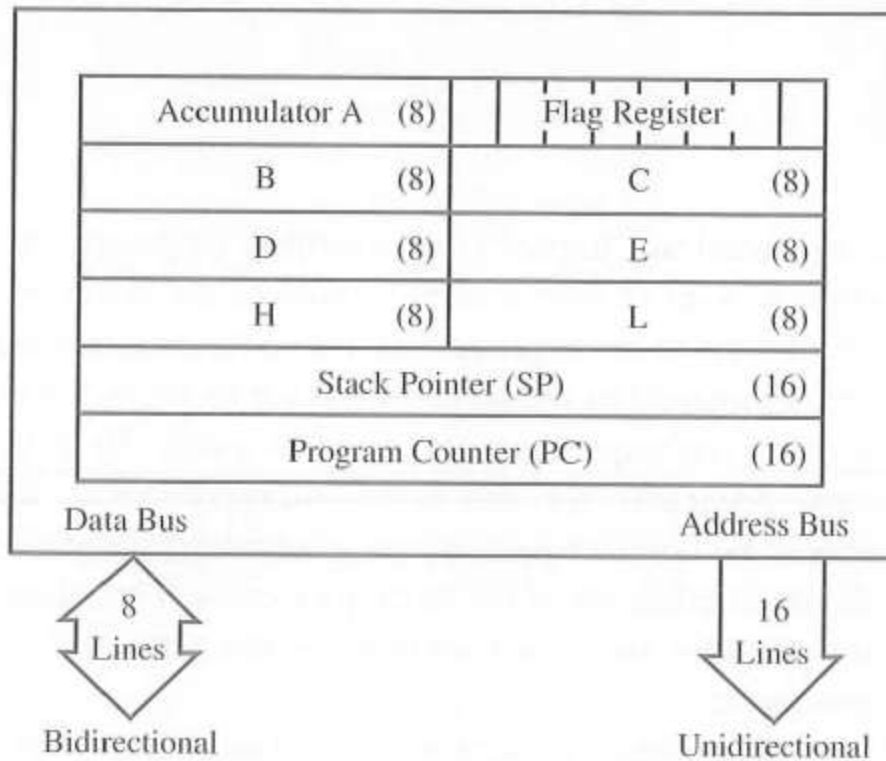
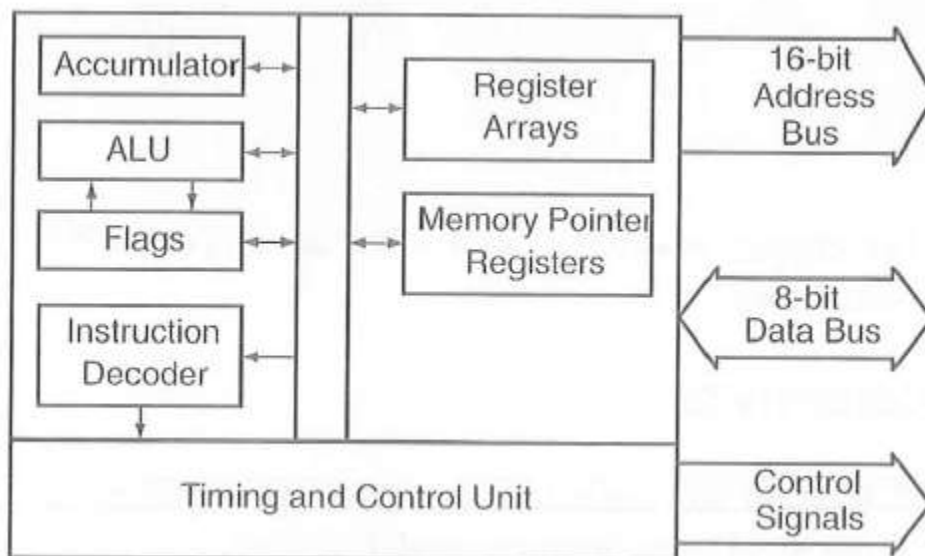


Fig: Programming model of 8085 microprocessor



The 8085/8080A has six general-purpose registers(GPR) to store 8-bit data; these are identified as B, C, D, E, H, and L as shown in the above figure. They can be combined as register pairs - BC, DE, and HL

- to perform some 16-bit operations. The programmer can use these registers to store or copy data into the registers by using data copy instructions

Program Counter (PC)

This 16-bit register deals with sequencing the execution of instructions. This register is a memory pointer. Memory locations have 16-bit addresses, and that is why this is a 16-bit register. The microprocessor uses this register to sequence the execution of the instructions. The function of the program counter is to point to the memory address from which the next byte is to be fetched. When a byte (machine code) is being fetched, the program counter is incremented by one to point to the next memory location

Stack Pointer (SP)

The stack pointer is also a 16-bit register used as a memory pointer. It points to a memory location in R/W memory, called the stack. The beginning of the stack is defined by loading 16-bit address in the stack pointer. The stack concept is explained in the chapter "Stack and Subroutines."

Instruction Register/Decoder

Temporary storage for the current instruction of a program. Latest instruction sent here from memory prior to execution. Decoder then takes instruction and 'decodes' or interprets the instruction. Decoded instruction then passed to next stage.

Memory Address Register

Holds address, received from PC, of next program instruction. Feeds the address bus with addresses of location of the program under execution.

Control Generator

Generates signals within microprocessor to carry out the instruction which has been decoded. In reality causes certain connections between blocks of the uP to be opened or closed, so that data goes where it is required, and so that ALU operations occur

Memory

Program, data and stack memories occupy the same memory space. The total addressable memory size is 64 KB.

Program memory - program can be located anywhere in memory. Jump, branch and call instructions use 16-bit addresses, i.e. they can be used to jump/branch anywhere within 64 KB. All jump/branch instructions use absolute addressing.

Data memory - the processor always uses 16-bit addresses so that data can be placed anywhere.

Stack memory is limited only by the size of memory. Stack grows downward.

- First 64 bytes in a zero memory page should be reserved for vectors used by RS instructions

Interrupts

INTR is maskable 8085 compatible interrupt. When the interrupt occurs the processor fetches from the bus one instruction, usually one of these instructions

One of the 8 RST instructions (RST0 - RST7). The processor saves current program counter into stack and branches to memory location $N * 8$ (where N 3-bit number from 0 to 7 supplied with the RST instruction).

RST5.5 is a maskable interrupt. When this interrupt is received the processor saves the contents of the PC register into stack and branches to 002CH (hexadecimal) address.

RST6.5 is a maskable interrupt. When this interrupt is received the processor saves the contents of the PC register into stack and branches to 0034H (hexadecimal) address.

RST7.5 is a maskable interrupt. When this interrupt is received the processor saves the contents of the PC register into stack and branches to 003CH (hexadecimal) address.

TRAP is a non-maskable interrupt. When this interrupt is received the processor saves the contents of the PC register into stack and branches to 0024H (hexadecimal) address.

All maskable interrupts RST 5.5, RST6.5 and RST7.5 can be enabled or disabled individually using SIM instruction.

Serial communication Signal

SID - Serial Input Data Line: The data on this line is loaded into accumulator bit 7 whenever a RIM instruction is executed.

SOD – Serial Output Data Line: The SIM instruction loads the value of bit 7 of the accumulator into SOD latch if bit 6 (SOE) of the accumulator is 1.

RESET IN

- When this signal goes low, the program counter (PC) is set to Zero, μp is reset and resets the interrupt enable and HLDA flip-flops
- The data and address buses and the control lines are tri-stated during RESET because of asynchronous nature of RESET, the processor internal registers and flags may be altered by RESET with unpredictable results
- Upon power-up, RESET IN must remain low for at least 10 ms after minimum Vcc has been reached.
- For proper reset operation after the power – up duration, RESET IN should be kept low a minimum of three clock periods.

RESET OUT:

This signal indicates that μp is being reset. This signal can be used to reset other devices. The signal is synchronized to the processor clock and lasts an integral number of clock periods

DMA Signals

HOLD:

Indicates that another master is requesting the use of the address and data buses. The CPU, upon receiving the hold request, will relinquish the use of the bus as soon as the completion of the current bus transfer.

- Internal processing can continue. The processor can regain the bus only after the HOLD is removed.
- When the HOLD is acknowledged, the Address, Data RD, WR and IO/M lines are tri-stated.

HLDA:

Hold Acknowledge: Indicates that the CPU has received the HOLD request and that it will relinquish the bus in the next clock cycle.

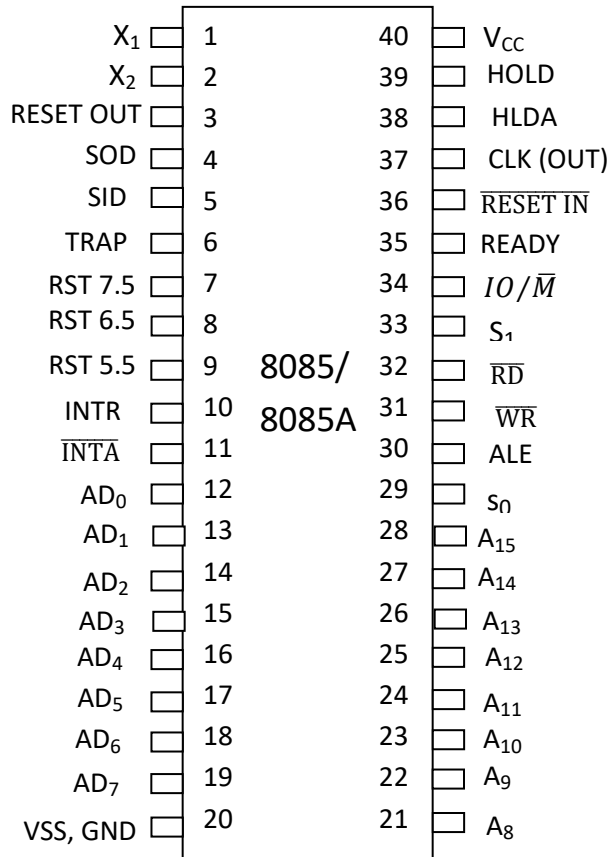
- HLDA goes low after the Hold request is removed. The CPU takes the bus one half-clock cycle after HLDA goes low.

READY:

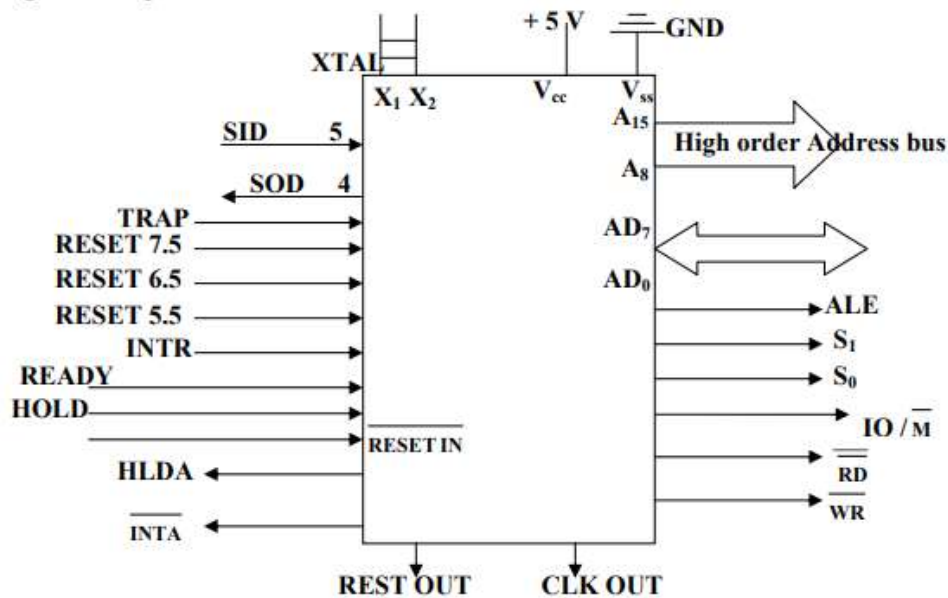
This signal Synchronizes the fast CPU and the slow memory, peripherals.

- If READY is high during a read or writes cycle, it indicates that the memory or peripheral is ready to send or receive data.
- If READY is low, the CPU will wait an integral number of clock cycle for READY to go high before completing the read or write cycle.
- READY must conform to specified setup and hold times.

8085 microprocessor Pin diagram

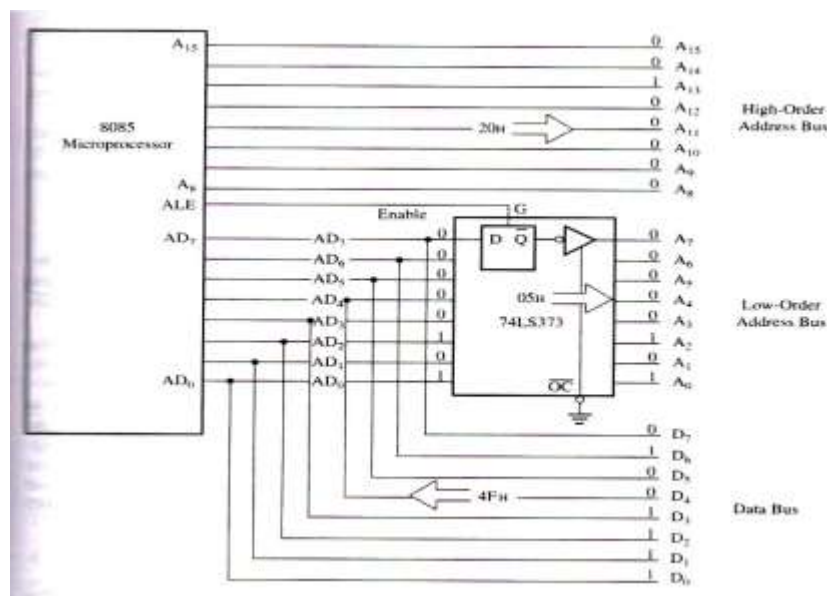


Signal Groups of 8085



AD₇ – AD₀ (Input/Output tri-state) Multiplexed Address/Data Bus; Lower 8 bits of the memory address (or I/O address) appear on the bus during the first clock cycle of a machine state. It then becomes the data bus during the second and third clock cycles. 3 states during Hold and Halt modes.

Demultiplexing address and data bus



ALE (Output) Address Latch Enable: It occurs during the first clock cycle of a machine state and enables the address to get latched into the on chip latch of peripherals. The falling edge of ALE is set to guarantee setup and hold times for the address information. ALE can also be used to strobe the status information. ALE is never tri-stated.

$IO/\overline{M}$: This is a status signal used to differentiate I/O and memory operations. When it is high it indicates I/O operations and low indicates memory operations. This signal is combined with $\overline{RD}$ and $\overline{WR}$ to generate I/O and memory control signals.

S_0, S_1 (Output) Data Bus Status. Encoded status of the bus cycle:

Machine Cycle	Status			Control Signals
	$IO/\overline{M}$	S_1	S_0	
Opcode fetch	0	1	1	$\overline{RD} = 0$
Memory Read	0	1	0	$\overline{RD} = 0$
Memory Write	0	0	1	$\overline{WR} = 0$
I/O Read	1	1	0	$\overline{RD} = 0$
I/O Write	1	0	1	$\overline{WR} = 0$
Interrupt Acknowledge	1	1	1	$\overline{INTA} = 0$
HALT	Z	0	0	$\overline{RD}, \overline{WR} = Z \text{ and } \overline{INTA} = 1$
HOLD	Z	X	X	$\overline{RD}, \overline{WR} = Z \text{ and } \overline{INTA} = 1$
RESET	Z	X	X	$\overline{RD}, \overline{WR} = Z \text{ and } \overline{INTA} = 1$

Read Signal ($\overline{RD}$) (Output); It is low active read control signal, this signal indicates that the selected memory or I/O or memory device is to be read and data are available on data bus..

Write Signal ($\overline{WR}$) (Output): It is low active write control signal; This indicates the data on the data bus to be written into the selected memory or I/O location.

Address Bus:

- The 8085 has 16 signal lines(pins) that are used as the address bus.
- These lines are breaks into two segments : ($A_{15} - A_8$) and : ($AD_7 - AD_0$)
- The eight lines, ($A_{15} - A_8$) are unidirectional and used for most significant bits, called the high order address of 16 bit address.
- The signal lines $AD_7 - AD_0$ are used for dual purpose.

Data Bus:

- This is used for the exchange of data between processor, memory and peripherals
- It is bidirectional so that it allows data flow in both directions
- The signal lines $AD_7 - AD_0$ are bidirectional: they serve a dual purpose. These lines are used as low-order address bus as well as data bus. The low order address bus can be separated from data signals using the latch.

Memory Range of 8085

- 8085 has 16-bit address bus ($A_0 - A_{15}$).
- It means, it can access 2^{16} memory locations.

$$2^{16} = 2^6 \times 2^{10} = 2^6 \times 1K = 64 \times 1K = 64K$$

So, 8085 can access up to 64K memory locations.

At every memory location, 8-bits i.e. 1-Byte is stored.

Hence, 8085 can access up to 64KB of memory data.

- The memory address range is

0000 0000 0000 0000B	0000H	00 00H
0000 0000 0000 0001B	0001H	00 01H
⋮	⋮	
1111 1111 1111 1110B	FFFEH	FF FEH
1111 1111 1111 1111B	FFFFH	FF FFH

8085 Interrupts:

A microprocessor has only two ways to determine the condition that exist in internal & external circuitry.

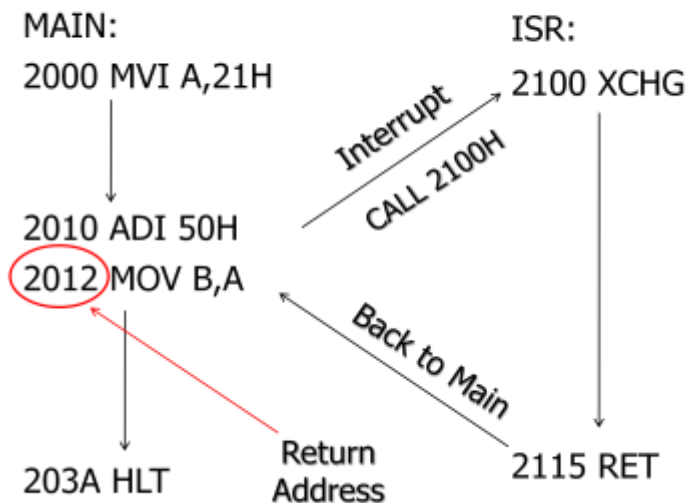
- Using Software, called Status Check or Polling.

Occupy the processor. Processor has to execute instructions continuously in a loop, until the desired state is identified.

- Using Interrupt.

Does not engage the processor. After initialization, device informs the processor that desired state is achieved. This information is given by a special signal called *interrupt*

Illustration of Interrupts



A processor can normally be interrupted by two methods, either by an instruction or by hardware. Accordingly the interrupts may be classified as

– Software Interrupts

- Execution of an instruction causes an interrupt to processor.

– Hardware Interrupts

- Application of an appropriate logic status (HIGH/LOW) on a hardware pin of processor causes an interrupt to processor.

Software Interrupts:

8085 has a special instruction “RST n” that can be used as software interrupt.

- RST n means *Restart* (RST) execution from a memory location addressed by the integer n.
- n is an integer that can vary from 0 to 7.
- Memory address given by n can be calculated as,
Address = $n * 8$.
- E.g. RST 3 Address = $3 * 8 = 24_D = 18_H = 0018_H$. So RST 3 means restart execution from memory location 0018_H .
- RST n is normally used as *Software Breakpoint Interrupt*.
- Such interrupts can be used to store status of program which is many times useful for debugging.
- Recall the use of RST 5 in Lab
- In order for processor to resume exactly from the same point where it has left it is necessary that last instruction of ISS should be *RET*.

RST n

- E.g. RST 3
- Calculate address for RST 3 i.e. 0018_H

(SP-1) $\leftarrow$ PCH(higher byte of program counter)

(SP-2) $\leftarrow$ PCL(lower byte of program counter)

SP $\leftarrow$ SP – 2

PC $\leftarrow$ 0018_H

- Execution of RST n takes 3 Machine cycles namely, Opcode Fetch, Memory Write and Memory Write.
- Opcode Fetch takes 6 T-States, since it not only fetches & decode the instruction, but also calculates the address for n.
- Rest of the two write cycles stores return address into the stack so that processor can resume its normal execution sequence.
- It is clear that, RST 3 means *Restart* (RST) execution from memory location 0018_H after storing return address in stack.
- *RST 3 can equivalently be said CALL 0018_H.*

Hence, RST n is sometimes, also called as *1-byte CALL instruction*

- Restart address 0018_H for RST 3 is also called *Vector Address* for RST 3 i.e. *Interrupt Vector Address* for RST 3 is 0018_H.
- In the similar analogy the restart number “n,” in RST n, is called *Interrupt Vector*.

Since both the *Interrupt Vector* and *Interrupt Vector Address* represents same thing, the memory address, the two terms are used synonymously

- A table comprising all the vector addresses for instruction RST n (n=0 to 7) is called *Interrupt Vector Table* and is shown next.

Interrupt Vector Table

Vector (n)	RST n Instruction	Vector Address
0	RST 0	0000 _H

1	RST 1	0008 _H
2	RST 2	0010 _H
3	RST 3	0018 _H
4	RST 4	0020 _H
5	RST 5	0028 _H
6	RST 6	0030 _H
7	RST 7	0038 _H

Hardware Interrupts:

8085 has several hardware pins through which, an interrupt request can be lodged. Below are list of interrupt signals offered by 8085

- TRAP (RST 4.5)
 - RST 7.5
 - RST 6.5
 - RST 5.5
 - INTR
 - **Interrupt Vector Table**
- ❖ Vector addresses for hardware interrupts are in boldface. Non boldface addresses are for software interrupts.
- ❖ Note that only 4-byte memory space is available between RST 4.5 & RST 5 and so on

Vector (n)	RST n Instruction	Vector Address
0	RST 0	0000 _H
1	RST 1	0008 _H
2	RST 2	0010 _H
3	RST 3	0018 _H
4	RST 4	0020 _H
4.5	TRAP RST 4.5	0024_H
5	RST 5	0028 _H
5.5	RST 5.5	002C_H
6	RST 6	0030 _H
6.5	RST 6.5	0034_H
7	RST 7	0038 _H
7.5	RST 7.5	003C_H

Interrupts can be divided in three categories

- Vectored Interrupts & Non-Vectored Interrupts
- Maskable Interrupts & Non-Maskable Interrupts
- Edge Triggered Interrupts & Level Triggered Interrupts

Vectored interrupts- These type of interrupts have fixed memory address :

Example: TRAP, RST7.5, RST6.5 , RST5.5 and RST 0 to RST 7

Non-vectored Interrupts: In this type of interrupt, the interrupt address is not known to the processor so; the interrupt address needs to be sent externally by the device to perform interrupts.

Example: INTR

Maskable Interrupts : These type interrupts can be disabled by writing instruction in program.

Example: RST7.5, RST6.5, RST5.5 , INTR

Non-Maskable Interrupts: These type interrupts cannot be disabled by writing instruction in program.

Example: TRAP

Edge Triggered Interrupts: TRAP, RST7.5

Level Triggered Interrupts: TRAP, RST6.5, RST5.5 and INTR

Interrupt	Priority	Trigger
TRAP	1(highest)	Edge and level triggered
RST7.5	2	Edge triggered
RST6.5	3	level triggered
RST5.5	4	level triggered
RST4.5	5(lowest)	level triggered

Interrupt Service Routine (ISR)

A small program or a routine that when executed, services the corresponding interrupting source is called an ISR.

TRAP

It is a non-maskable interrupt, having the highest priority among all interrupts. By default, it is enabled until it gets acknowledged. In case of failure, it executes as

ISR and sends the data to backup memory. This interrupt transfers the control to the location 0024H.

RST7.5

It is a maskable interrupt, having the second highest priority among all interrupts. When this interrupt is executed, the processor saves the content of the PC register into the stack and branches to 003CH address.

RST 6.5

It is a maskable interrupt, having the third highest priority among all interrupts. When this interrupt is executed, the processor saves the content of the PC register into the stack and branches to 0034H address.

RST 5.5

It is a maskable interrupt. When this interrupt is executed, the processor saves the content of the PC register into the stack and branches to 002CH address.

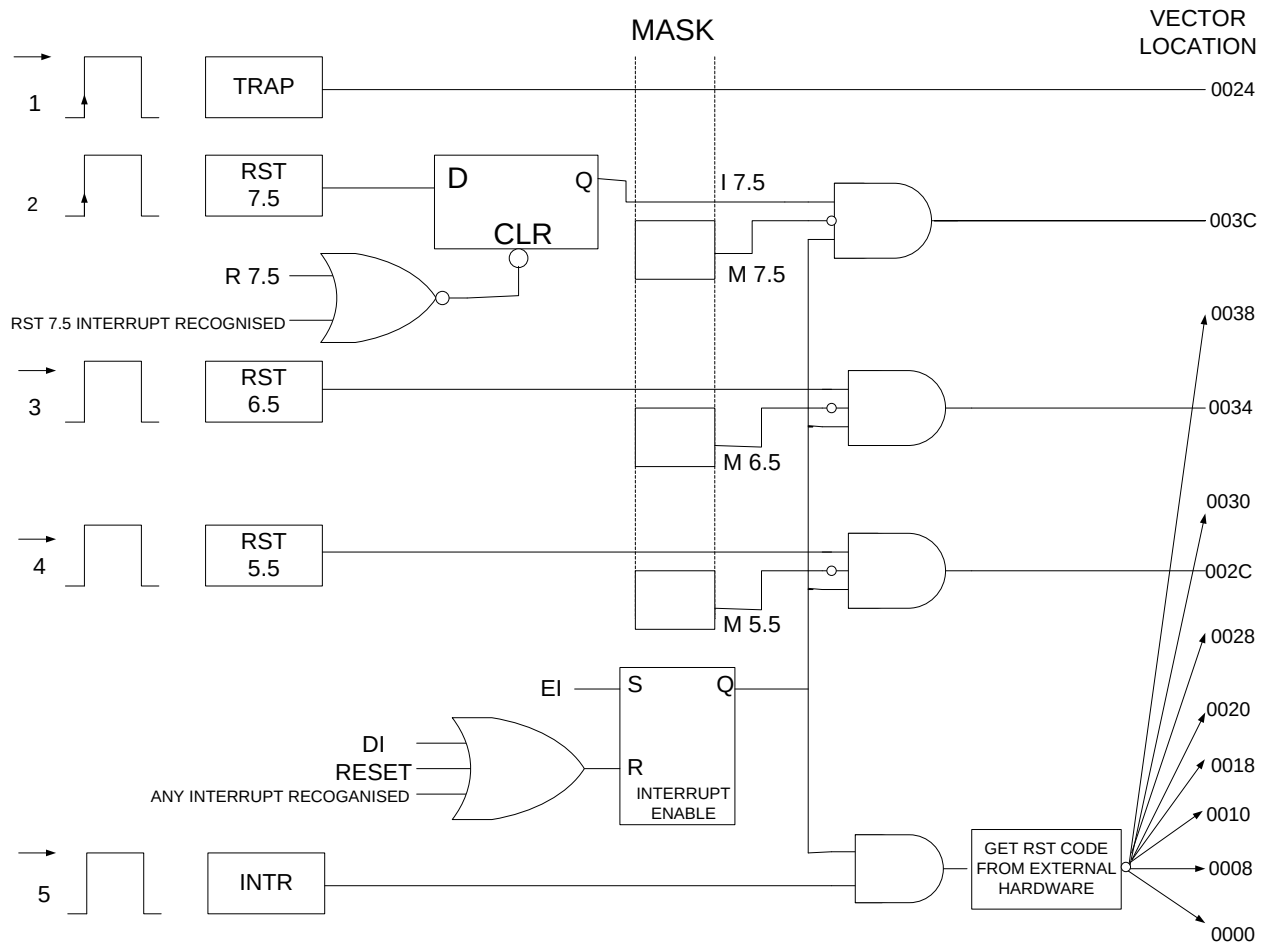
INTRs

It is a maskable interrupt, having the lowest priority among all interrupts. It can be disabled by resetting the microprocessor.

When INTR signal goes high, the following events can occur –

- The microprocessor checks the status of INTR signal during the execution of each instruction.
- When the INTR signal is high, then the microprocessor completes its current instruction and sends active low interrupt acknowledge signal.
- When instructions are received, then the microprocessor saves the address of the next instruction on stack and executes the received instruction.

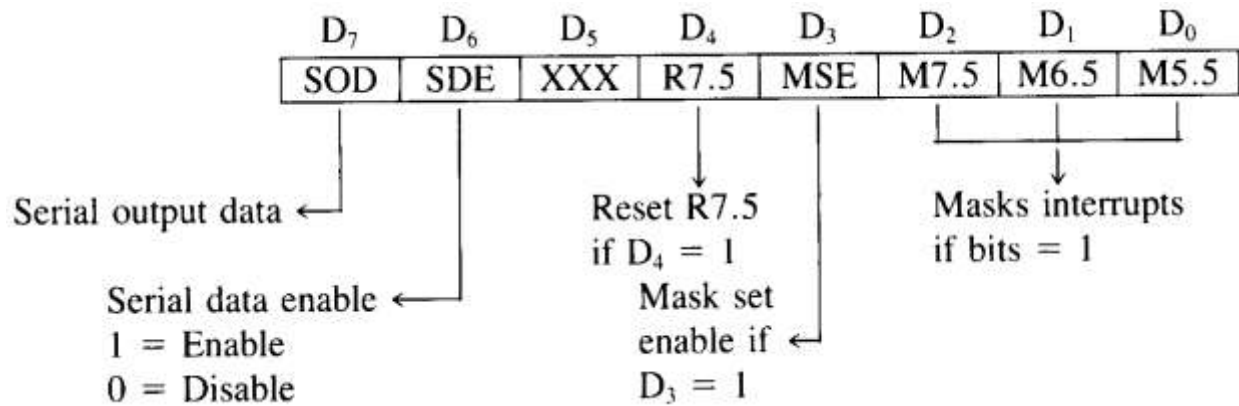
Interrupt Structure



SIM Instruction:

Opcode	Operand	Description
SIM	None	Set Interrupt Mask

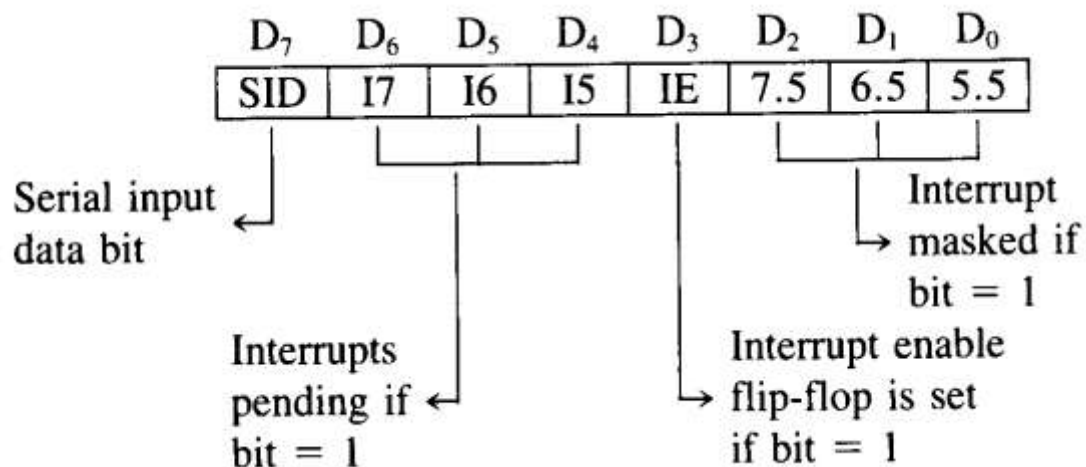
- This is a multipurpose instruction and used to implement the 8085 interrupts RST7.5, RST6.5, RST5.5, and serial data output.
- The contents of SIM register is called the SIM word.
- The instruction interprets the accumulator contents as given below.
- Example: **SIM**



RIM Instruction:

Opcode	Operand	Description
RIM	None	Read Interrupt Mask

- This is a multipurpose instruction used to read the status of pending interrupts RST7.5, RST6.5, RST5.5 and read serial data input bit.
- The instruction loads eight bits in the accumulator with the following interpretations.
- Example: **RIM**



Instruction Set

Each instruction has two parts.

- The first part is the task or operation to be performed.
 - This part is called the “opcode” (operation code).
- The second part is the data to be operated on
 - Called the “operand”.

Instruction set of 8085 microprocessor

- Data Transfer Instruction
- Arithmetic Instructions
- Logical Instructions
- Branching Instructions
- Control Instructions

Data transfer instructions

- These instructions move data between registers,
- between memory and registers.
- Data between a memory location and a register
- Data between an I\O Device and the accumulator.
- These instructions copy data from source to destination.
- While copying, the contents of source are not modified.

Example: MOV, MVI, IN, OUT, LDA, STA, LDAX, STAX, LXI, LHLD,SHLD, XCHG

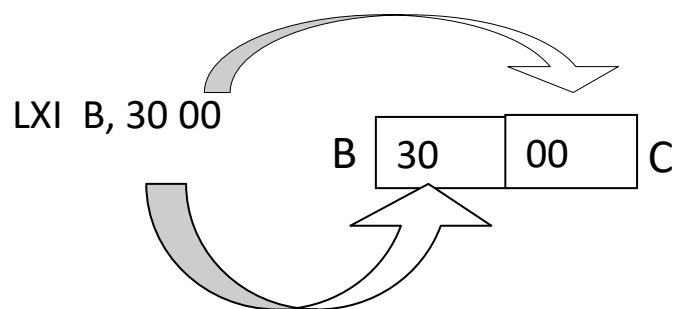
Opcode	Operand	Description
MOV	Rd, Rs	Copy from source to destination.
	M, Rs	All MOV instructions are of 1-byte instruction
	Rd, M	
MOV	Rd, Rs	- Copy the contents of source register to destination register Rd or Rs : A,B,C,D,E,H,L - The contents of the source register are not altered.
Ex. MOV A, B; MOV B,C ; MOV E,A etc		
MOV	M, Rs	Copy the contents of source register(Rs) to memory specified by HL pair register.
MOV	M, A	Copy the contents of accumulator to the memory address specified by HL pair register
MOV	Rd, M	Copy the contents of memory location specified by HL pair register to the destination register(Rd) -The 16-bit contents of the HL register pair are treated as a 16-bit address and used to identify the memory location. -The HL register pair is always used in conjunction with the memory register "M"
MOV	B, M	Copy the contents of memory specified by HL pair register to the register B
MVI	Rd, 8-bit M, 8-bit	Move Immediate 8 bit data
MVI	A, 8-bit	2-byte instruction - Copy the 2nd byte of 8 bit-data to the accumulator
MVI	M, 8-bit	2-byte instruction - Copy the 2nd byte of 8-bit data to the accumulator
OUT	8-bit port	-Output to the port

	address	-1-byte instruction Copies the contents of the accumulator to the output port address
IN	8-bit port address	-Input from the port -1-byte instruction Copies the 2 nd byte from the port address to the accumulator

LDA	16-bit address	Load accumulator - 3-byte instruction -Copies the contents of memory location specified by 16-bit address - the contents of source is not changed
Example: LDA 2500H		
LDAX	B/D register pair	Load accumulator direct - 1-byte instruction - This instruction only uses the BC or DE pair. – It does not accept the HL pair
LDAX	B	Copy the 8-bit contents of the memory location specified by the BC register pair into the Accumulator
LDAX	D	-copies the contents of memory location specified by DE pair register into the accumulator -The contents of the register pair and the memory location are not changed
LXI	R _p , 16-bit data	Load register pair(R _p) immediate -3-byte instruction -loads 16-bit data in the specified register pair

LXI	B, 16-bit data	Load 16-bit number immediate in BC pair register
LXI	D, 16-bit data	Load 16-bit number immediate in DE pair register
LXI	H, 16-bit data	Load 16-bit number immediate in HL pair register
LXI	SP, 16-bit data	Load 16-bit address immediate in stack pointer register

The upper two digits are placed in the 1st register of the pair and the lower two digits(8-bits) in the 2nd register.



Opcode	Operand	Description
STA	16-bit address	Store Accumulator Direct
e.g. STA 3502H; store the contents of accumulator to the memory location 3502H		
STAX	Reg. Pair	Store accumulator indirect
STAX	B	Copy the contents of accumulator into the memory location specified by the contents of the B,C register pair.
STAX	D	Copy the contents of accumulator into the memory location specified by the contents of

		the D,E register pair.
--	--	------------------------

Opcode	Operand	Description
LHLD	16-bit address	Load H-L register pair direct • 3- byte instruction • It Copies the contents of the memory location specified by 16-bit address to register L and copies the contents of next memory location in register H. *The contents of source memory location is not altered.
SHLD	16-bit address	Store H-L register pair direct • 3- byte instruction • Store contents of register L to the memory location specified by the 16-bit address in the operands , and contents of H-register are stored in next memory location by increasing the operand. • The contents of registers H,L are not changed.
XCHG	None	-Exchange H and L with D and E -1-byte instruction The contents of register H are exchanged with D and contents of L are exchanged with contents of E register.
SPHL	None	Copy H-L pair to the Stack Pointer (SP) -This instruction loads the contents of H-L pair into SP register.
XTHL	None	Exchange H–L with top of stack - The contents of L register are exchanged with

		the location pointed out by the contents of the SP. -The contents of H register are exchanged with the next location (SP + 1).
PUSH	Reg. pair	Push register pair onto stack
PUSH	B	-The contents of register pair are copied onto stack.
PUSH	D	-SP is decremented and the contents of high-order registers (B, D, H, A) are copied into stack.
PUSH	H	-SP is again decremented and the contents of low-order registers (C, E, L, Flags) are copied into stack.
PUSH	SP	
POP	Reg. pair	Pop stack to register pair
POP	B	-The contents of top of stack are copied into register pair.
POP	D	-The contents of location pointed out by SP are copied to the low-order register (C, E, L, Flags).
POP	H	-SP is incremented and the contents of location are copied to the high-order register (B, D, H, A).
POP	SP	

Arithmetic Operations

These instructions perform the following arithmetic operations:

- Addition
- Subtract

- Increment
- Decrement

Opcode	operands	Description
ADD	R/M	Add register or memory to accumulator -Add contents of register(R) or memory(M) to the contents of accumulator -The result is stored in accumulator. -If the operand is memory location, its address is specified by H-L pair. Example: ADD C or ADD M
ADI	8-bit data	Add immediate to accumulator -The 8-bit data is added to the contents of accumulator. -The result is stored in accumulator. -All flags are modified to reflect the result of the addition. Example: ADI 35H
ACI	8-bit data	Add immediate to accumulator with carry -The 8-bit data and the Carry Flag (CY) are added to the contents of accumulator. -The result is stored in accumulator. -All flags are modified to reflect the result of the addition
ADC	R/M	Add register or memory to accumulator with carry -The contents of register or memory and Carry Flag (CY) are added to the contents of accumulator. -The result is stored in accumulator.

-If the operand is memory location, its address is specified by H-L pair

-All flags are modified to reflect the result of the addition

Example: ADC D or ADC M

DAD. Reg. Pair **Double addition**

-Add register pair to H and L register

-the 16-bit contents of specified register pair are added to the contents of the HL register pair and sum is saved in the HL register pair.

-The contents of source registers are not altered

DAA None **Decimal Adjust Accumulator**

-1-byte instruction

-The contents of accumulator are changed from binary value to two 4-digit binary coded decimal(BCD) digits.

-This is only instruction that uses auxiliary flag(internally) to perform the binary to BCD conversion.

Example:

- **DAD R_p HL ← HL + R_p**
 - Only instruction that performs 16-bit addition.
 - Only CY Flag is affected.
 - Takes 3 machine cycles for execution.

E.g.

- **DAD B HL ← HL + BC**

- DAD D HL $\leftarrow$ HL + DE
- DAD H HL $\leftarrow$ HL + HL
- DAD SP HL $\leftarrow$ HL + SP

DAA – Decimal Adjust Accumulator

- **DAA A $\leftarrow$ BCD (A) after ADD**
 - Used after ADD instruction.
 - Checks both the Nibbles of Accumulator
 - Lower Nibble > 9 **OR** Flag AC=1 then
Lower Nibble $\leftarrow$ Lower Nibble + 6
 - Higher Nibble > 9 **OR** Flag CY=1 then
Higher Nibble $\leftarrow$ Higher Nibble + 6

FLAGS: All the flags are affected.

Opcode operands Description

SUB R/M Subtract register or memory from accumulator

- Subtract contents of register(R) or memory(M) from the contents of accumulator
- The result is stored in accumulator.
- If the operand is memory location, its address is specified by H-L pair.
- All flags are modified to reflect the result of subtraction

Example: SUB C or SUB M

SUI 8-bit data Subtract immediate from accumulator

- The 8-bit data is subtracted from the contents of accumulator.
- The result is stored in accumulator.

-All flags are modified to reflect the result of the subtraction

Example: SUI 35H

SBB R/M

Subtract register or memory from accumulator with borrow

-The contents of the register or memory location and Borrow Flag (i.e. CY) are subtracted from the contents of the accumulator.

-The result is stored in accumulator.

-If the operand is memory location, its address is specified by H-L pair.

-All flags are modified to reflect the result of subtraction.

Example: SBB B or SBB M

SBI 8-bit data

Subtract immediate from accumulator with borrow

-The 8-bit data and the Borrow Flag (i.e. CY) is subtracted from the contents of the accumulator.

-The result is stored in accumulator.

-All flags are modified to reflect the result of subtraction.

Example: SBI 55 H

INR R/M

Increment register or memory by 1

-The result is stored in the same place.

-If the operand is a memory location, its address is specified by the contents of H-L pair.

INX Reg.pair

Increment register pair by 1

-The contents of register pair are incremented by 1.

-The result is stored in the same place.

Example: INX H

DCR	R/M	Decrement register or memory by 1 -The result is stored in the same place. -If the operand is a memory location, its address is specified by the contents of H-L pair.
DCX	Reg.pair	Decrement register pair by 1 -The contents of register pair are decremented by 1. -The result is stored in the same place. Example: DCX SP

- **INR R – Increment the contents of specified register by 1.**
 - $R \leftarrow R + 1$
 - R can be any of the general purpose registers (A, B, C, D, E, H, L).
 - Affects all the flags except CY.
 - Register Addressing Mode.
 - 1-byte instruction and takes 1 Machine Cycle for execution.
 - E.g. INR B, INR A, INR C, INR D, INR E, INR H, INR L
 - Amounts to 7 different Opcodes.
- **DCR R – Decrement the specified register by 1.**
 - $R \leftarrow R - 1$
 - R can be any of the general purpose registers (A, B, C, D, E, H, L).
 - Affects all the flags except CY.
 - Register Addressing Mode.
 - 1-byte instruction and takes 1 Machine Cycle for execution.
 - E.g. DCR C, DCR H.
 - Amounts to 7 different Opcodes.
- **INR M – Increment the content of memory (location specified by HL Pair) by 1.**
 - $((HL)) \leftarrow ((HL)) + 1$
 - Affects all the flags except CY.
 - Indirect Addressing Mode.
 - 1-byte instruction

- Takes 3 Machine Cycles (Fetch, Read and Write) for execution.
- **DCR M – Decrement the indirect byte by 1 (memory location specified by HL Pair).**
 - $((HL)) \leftarrow ((HL)) - 1$
 - Affects all the flags except CY.
 - Indirect Addressing Mode.
 - 1-byte instruction
 - Takes 3 Machine Cycles (Fetch, Read and Write) for execution.

Logic Operations

These instructions perform logical operations on data stored in registers, memory and status flags.

*All operations are performed in context to Accumulator.

- The logical operations are:

- AND
- OR
- XOR
- Rotate
- Compare
- Complement

–ANA, ANI, ORA, ORI, XRA , XRI, RAL, RLC, RAR, RRC, CMP, CPI, CMA

Source: Accumulator and

- An 8-bit number
- The contents of a register
- The contents of a memory location

- **Destination:** Accumulator

Carry & Auxiliary carry flags are fixed and don't convey useful information.

- NOT => CMA
- Bitwise AND => ANA
- Bitwise OR => ORA
- Bitwise XOR => XRA

Logical instructions

Opcode	operands	Description
CMA	NONE	Complement the contents of ACC.
ANA	R/M	AND Accumulator with Register/Memory
ANI	8-bit	AND Accumulator with an 8-bit number
ORA	R/M	OR Accumulator with Register/Memory
ORI	8-bit	OR Accumulator with an 8-bit number
XRA	R/M	XOR Accumulator with Register/Memory
XRI	8-bit	XOR Accumulator with an 8-bit number

Explanation:

- **CMA $A \leftarrow \text{NOT } (A)$**
 - None of the Flags are affected.
 - Only Accumulator is complemented.
 - E.g. Let Flags are SF=0, ZF=0, PF=1, AC=1, CY=0 and A = 59H

A = 59H = 0101 1001

After CMA, A = 1010 0110 = A6H

FLAGS: SF=0, ZF=0, PF=1, AC=1, CY=0

Flags are same as previous i.e. unaffected.

LOGICAL AND

- **ANA R** ; A ← A AND R
 - Ex. **ANA D** ; A ← A AND D
- **ANA M** ; A ← A AND ((HL))
- **ANI DATA8** ; A ← A AND DATA8
 - Ex. **ANI 80H** A ← A AND 80H
- CY=0, AC=1, Rest of the flags are affected by the result.

e.g. ANI F0H A ← A AND F0H

ANI F0H, would MASK the Lower nibble of A.

Let A = 45H = 0100 0101

F0H = 1111 0000

After AND^{ing} A = 0100 0000 = 40H

Lower Nibble 0101(05) is Masked.

- **FLAGS:** SF = 0, ZF=0, PF=0.

LOGICAL OR

- **ORA R** ; A ← A OR R

- E.g. **ORA D** ; $A \leftarrow A \text{ OR } D$
- **ORI DATA8** ; $A \leftarrow A \text{ OR } DATA8$
 - E.g. **ORI 75H** ; $A \leftarrow A \text{ OR } 75H$
- **ORA M** ; $A \leftarrow A \text{ OR } ((HL))$
- CY=0, AC=0, Rest of the flags are affected by the result.

LOGICAL XOR

- **XRA R** ; $A \leftarrow A \text{ XOR } R$
 - E.g. **XRA D** ; $A \leftarrow A \text{ XOR } D$
- **XRI DATA8** ; $A \leftarrow A \text{ XOR } DATA8$
 - E.g. **XRI 95H** ; $A \leftarrow A \text{ XOR } 95H$
- **XRA M** ; $A \leftarrow A \text{ XOR } ((HL))$
- CY=0, AC=0, Rest of the flags are affected by the result.

e.g. **XRI FFH** ; $A \leftarrow A \text{ XOR } FFH$

XRI FFH, would Complement the content of A.

Let $A = 45H = 0100\ 0101$

$FFH = 1111\ 1111$

After XOR^{ing} $A = 1011\ 1010 = BAH$

Accumulator is Complemented.

- **FLAGS:** SF = 1, ZF=0, PF=0.

Masking of bits:

- The AND instruction can be used to clear specific destination bits× while preserving others.

- A 0 mask bit clears the corresponding destination bit; a 1 mask bit× preserves the corresponding destination bit.
- The OR instruction can be used to set specific destination bits× while preserving the others.
- A 1 mask bit sets the corresponding destination bit; a 0 mask bit× preserves the corresponding destination bit.
- The XOR instruction can be used to complement specific× destination bits while preserving the others.
- A 1 mask bit complements the corresponding destination bit; a 0× mask bit preserves the corresponding destination bit.

Complement:

- 1's complement of the contents of the accumulator.

CMA No operand

- **Compare**

- Compare the contents of a register or memory location with the contents of the accumulator for equality, greater than or less than.

CMP R/M Compare the contents of the register or memory location to the contents of the accumulator.

- if $(A) < (\text{register}(R))/\text{memory}(M)$): carry flag is set
- if $(A) = (R/M)$: zero flag is set
- if $(A) > (R/M)$: carry and zero flags are reset.

CPI 8-bit Compare the 8-bit number to the contents

of the accumulator.

- The compare instruction sets the flags (Z, CY, and S).
- The compare is done using an internal subtraction that does not change the contents of the accumulator. $A - (R / M / 8\text{-bit data})$
- if $(A) < \text{data}$: carry flag is set
- if $(A) = \text{data}$: zero flag is set
- if $(A) > \text{data}$: carry and zero flags are reset
- **Example:** CPI 29H

e.g. **CMP B ;FLAG $\leftarrow A - B$**

- Compare register B with Accumulator
- It performs a subtraction to compare
- Result of the subtraction is not stored i.e. after CMP, both A & B remains unaffected.
- Result of subtraction is reflected through the *Flags* i.e. all the flags are affected by CMP

Example:

Let initially **A = 45H and B = 30H**

Execution of CMP B means

- Perform $A - B = 45_H - 30_H = 15_H = (0001\ 0101)_B$

After Execution of CMP B

- $A = 45_H$ and $B = 30_H$
- *Flags* SF=0, ZF=0, PF=0, AC=0, CY=0.
- Note that both A & B, remains unaffected from compare but all the flags are affected.

Uses

$CY=1 \Rightarrow A < B$,

$CY=0 \Rightarrow A \geq B$,

$(CY=0 \text{ AND } ZF=0) \Rightarrow A > B$

- **CMP R** $FLAGS \leftarrow A - R$
– E.g. **CMP D** $FLAGS \leftarrow A - D$
- **CPI DATA8** $FLAGS \leftarrow A - DATA8$
– E.g. **CPI 80H** $FLAGS \leftarrow A - 80H$
- **CMP M** $FLAGS \leftarrow A - ((HL))$

Additional Logic Operations

- Rotate the contents of the accumulator one position to the left or right.

RLC Rotate the accumulator left.
Bit 7 goes to **bit 0 AND** the **Carry flag**.

RAL Rotate the accumulator left through the carry.
Bit 7 goes to the **carry** and **carry goes to bit 0**

RRC Rotate the accumulator right.
Bit 0 goes to **bit 7 AND** the **Carry flag**.

RAR Rotate the accumulator right through the carry.
Bit 0 goes to the **carry** and **carry goes to bit 7**.

Rotating Accumulator(Acc)

- **RAR** **Rotate Acc Right through Carry**

$CY \Rightarrow A_7, A_7 \Rightarrow A_6, \dots, A_1 \Rightarrow A_0, A_0 \Rightarrow CY$



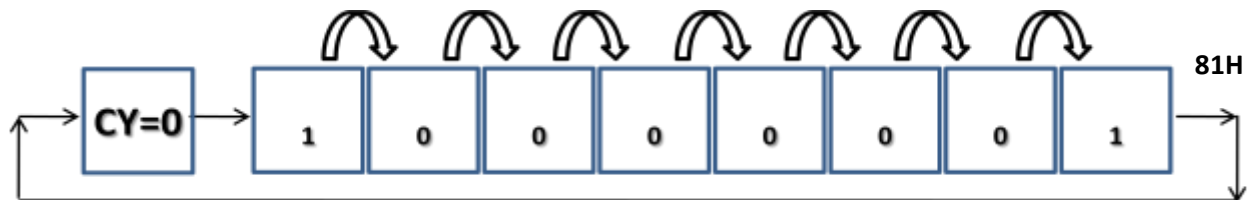
- **RRC** **Rotate Acc Right without Carry**
 $A_7 \Rightarrow A_6, \dots, A_1 \Rightarrow A_0, A_0 \Rightarrow A_7, A_0 \Rightarrow CY$



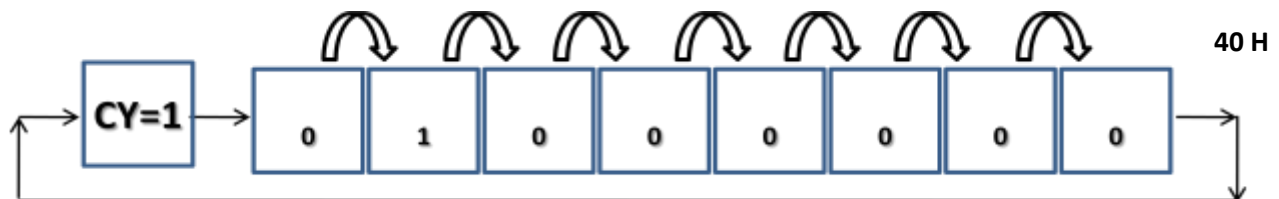
Example:

- RAR** **Rotate Acc Right through Carry**
 $CY \Rightarrow A_7, A_7 \Rightarrow A_6, \dots, A_1 \Rightarrow A_0, A_0 \Rightarrow CY$

Before Instruction ; A= 81H



After RAR A Instruction

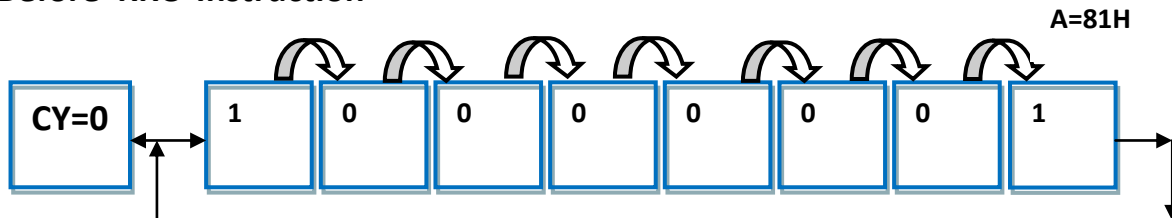


RRC **Rotate Acc Right without Carry**

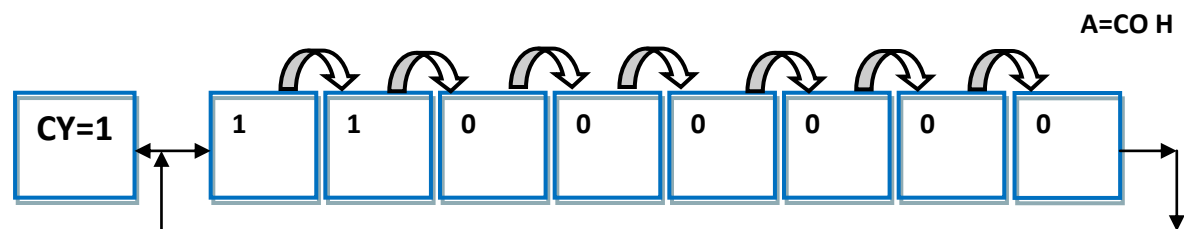
- Each binary bit of the accumulator is rotated right by one bit position

$A_7 \Rightarrow A_6, \dots, A_1 \Rightarrow A_0, A_0 \Rightarrow A_7, A_0 \Rightarrow CY$

Before RRC Instruction



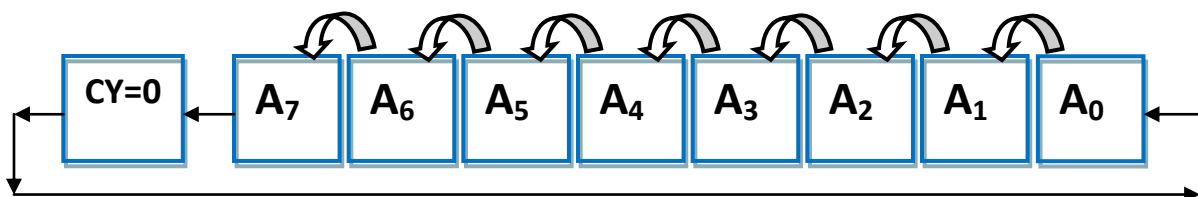
After 1st RRC Instruction



Rotating Accumulator

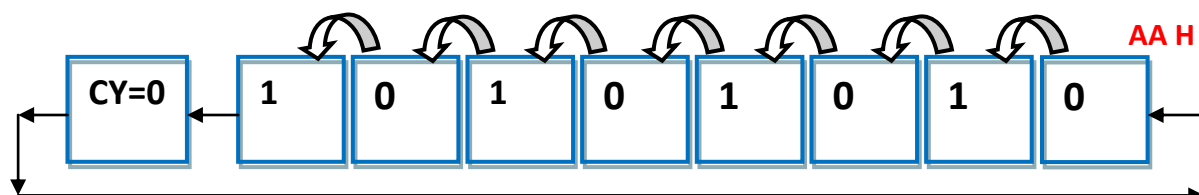
- **RAL** Rotate Acc Left through Carry

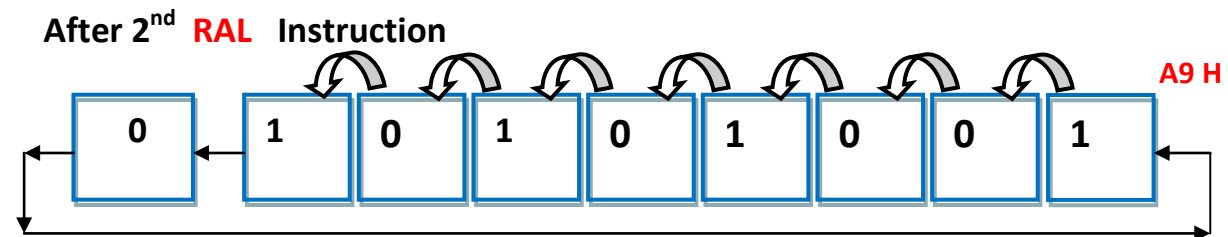
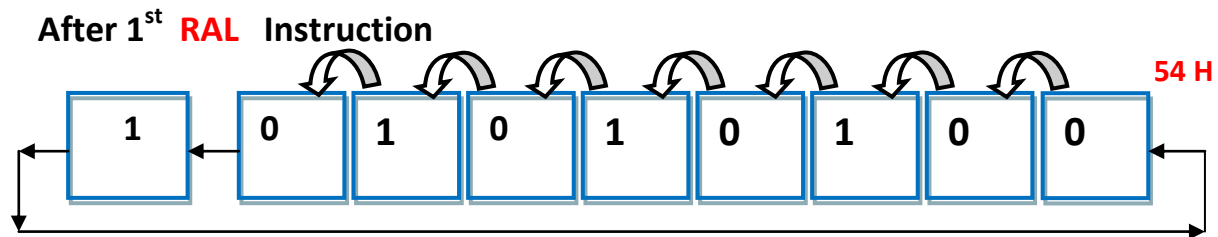
$CY \leftarrow A_7, A_7 \leftarrow A_6, \dots, A_1 \leftarrow A_0, A_0 \leftarrow CY$



Example

Before RAL Instruction

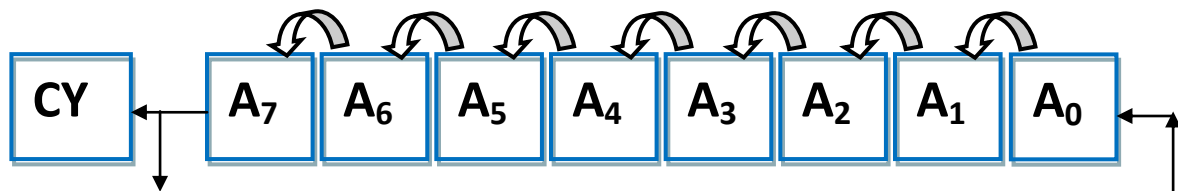




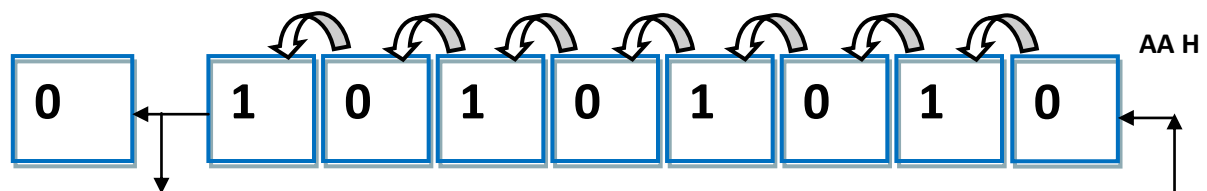
• **RLC Rotate Acc Left without Carry**

- Each binary bit of the accumulator is rotated left by one position.

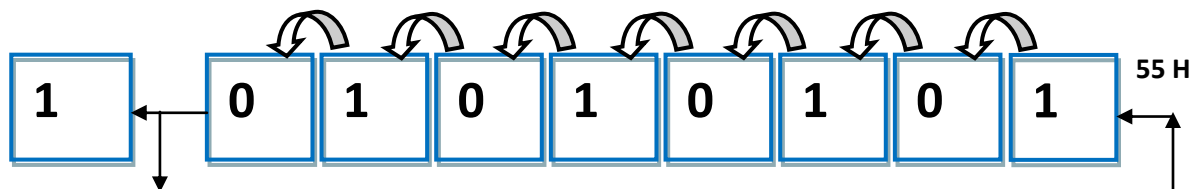
$$CY \leftarrow A_7, A_7 \leftarrow A_6, \dots, A_1 \leftarrow A_0, A_0 \leftarrow A_7$$



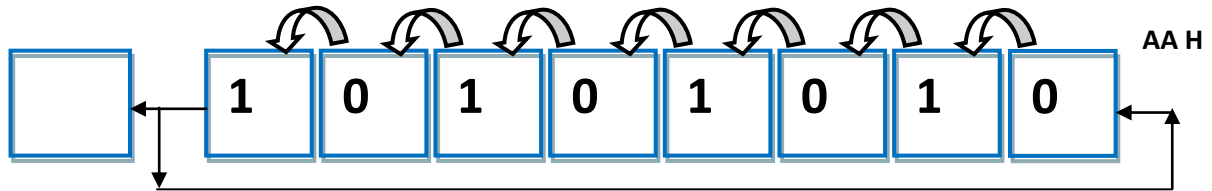
Before **RLC** Instruction



After 1st **RLC** Instruction



After 2nd **RLC** Instruction



Opcode	operands	Description
CMC	None	Complement carry

- The Carry flag is complemented.
- No other flags are affected.

Example: CMC.

Opcode	operand	Description
STC	None	Set the carry flag

- The Carry flag is set to 1.
- No other flags are affected.

Example: STC.

Branch Instructions

Opcode	operands	Description
JMP	16-bit address	- 3-byte instruction -The 2nd and 3rd bytes specify the 16-bit memory address

- **JMP** – Jump (Go) to an instruction to execute.

E.g. JMP LABEL

- This instruction is used to change the execution sequence.
- LABEL is a symbol placed before instruction where processor has to jump.

- E.g. JMP HERE
- Next is a simple example for understanding JMP
- **JMP ADDRESS** – Jump (Go) to ADDRESS to execute an instruction.
 - ADDRESS is actually the memory address where instruction INR A (Last Slide) is stored in memory.
 - E.g. JMP 5000H
 - $PCH \leftarrow 50H$ and $PCL \leftarrow 00H$, finally $PC=5000H$, showing address of instruction to be executed.
 - It takes 3 Machine Cycles for execution.

Conditional Jumps

- The conditional jump instructions allow the microprocessor to take decisions based on certain conditions indicated by flags
- The jump instruction used only four flags
Carry flag, Zero flag, Sign flag and Parity flag
- Two jump instructions are associated with each flag

Opcode	Operand	Description
JC	16-bit	Jump if a Carry Generated($CF=1$) i.e. Jump on Carry($CF = 1$).
JNC	16-bit	Jump if Carry Not Generated i.e. Jump on No carry($CF = 0$)
JZ	16-bit	Jump on Zero($ZF=1$)
JNZ	16-bit	Jump on No Zero($ZF=0$)
JP	16-bit	Jump if Result is Positive i.e. Jump on $SF = 0$
JM	16-bit	Jump if Result is Minus (Negative), i.e. Jump on $SF = 1$.
JPE	16-bit	Jump on EVEN Parity or Jump if Parity is SET, i.e. Jump on $PF = 1$.
JPO	16-bit	Jump on ODD Parity, or Jump if Parity is NOT SET

i.e. Jump on PF = 0.

***All instructions transfer the program to a memory location**

Explanation:

- **JZ** – Jump (Go) to instruction to execute if ZF=1.

E.g. JZ ADDRESS (Memory Address)

- Executes similar to JMP but checks ZF before jump. If ZF=1 then jump otherwise don't jump.

- **JNZ**– Jump (Go) to instruction to execute if ZF=0.

E.g. JNZ ADDRESS (Memory Address)

- Executes similar to JZ but checks for ZF=0 before jump. If ZF=0 then jump otherwise don't jump.

Opcode	operand	Description
CALL	16-bit address	Call unconditionally
- Used to CALL subroutine -The program sequence is transferred to the memory location specified by the 16-bit address given in the operand. -Before the transfer, the address of the next instruction after CALL (the contents of the program counter) is pushed onto the stack.		

Example: CALL 2025 H.

Opcode	operand	Description
Cx	16-bit address	Call conditionally
-The program sequence is transferred to the memory location specified by the 16-bit address given in the operand based on the specified flag of the PSW.		

-Before the transfer, the address of the next instruction after the call (the contents of the program counter) is pushed onto the stack

Opcode	Operand	Description
CC	16-bit address	CALL a subroutine if a Carry Generated(CF=1) i.e. CALL on Carry(CF = 1).
CNC	16-bit address	Call a subroutine if Carry Not Generated i.e. Call on No carry(CF = 0)
CZ	16-bit address	Call a subroutine on Zero i.e Call on (ZF=1)
CNZ	16-bit address	Call a subroutine on No Zero i.e. (ZF=0)
CP	16-bit address	Call a subroutine if Result is Positive i.e. Call on SF = 0
CM	16-bit address	Call a subroutine if Result is Minus(Negative), i.e. Call on SF = 1.
CPE	16-bit address	Call a subroutine on EVEN Parity or Call if Parity is SET,i.e. Call on PF = 1.
CPO	16-bit address	Call a subroutine on ODD Parity, or Call if Parity is NOT SET i.e. Call on PF = 0.

Return Instruction

Opcode	Operand	Description
RET	None	Return unconditionally

-The program sequence is transferred from the subroutine to the calling program.

-The two bytes from the top of the stack are copied into the program counter, and program execution begins at the new address

Opcode	operand	Description
Rx	16-bit address	Return conditionally

-The program sequence is transferred from the subroutine to the calling program based on the specified flag of the PSW.

-The two bytes from the top of the stack are copied into the program counter, and program execution begins at the new address.

Opcode	Operand	Description
RC	16-bit	Return from a subroutine if a Carry Generated(CF=1)i.e. Return on Carry(CF = 1).
RNC	16-bit	Return from a subroutine if Carry Not Generated i.e. Return on No carry(CF = 0)
RZ	16-bit	Return from a subroutine on Zero(ZF=1)
RNZ	16-bit	Return from a subroutine on No Zero(ZF=0)
RP	16-bit	Return from a subroutine if Result is Positive i.e. Return on SF = 0
RM	16-bit	Return from subroutine if Result is Minus (Negative), i.e. Return on SF = 1.
RPE	16-bit	Return from subroutine on EVEN Parity or Return if Parity is SET, i.e. Return on PF = 1.
RPO	16-bit	Return from subroutine on ODD Parity, or return if Parity is NOT SET i.e. return on PF=0

Stack, I/O and Machine Control Operations

- IN port-address. ;Input to accumulator from I/O port
[A] <-- [Port]
- OUT port-address ;Output from accumulator to I/O port

[Port] <-- [A]

- PUSH rp ;Push the content of register pair onto stack
- PUSH PSW ;PUSH Processor Status Word
- POP rp ;POP(retrieve) the content of register pair, which is saved, from the stack
- POP PSW ;Pop Processor Status Word
- HLT No operand (Halt)
- XTHL No operand ;Exchange stack-top with H-L
- SPHL ;Move the contents of H-L pair to stack pointer
- EI ;Enable Interrupts
- DI ;Disable Interrupts
- SIM ;Set Interrupt Masks
- RIM ;Read Interrupt Masks
- NOP ;No Operation

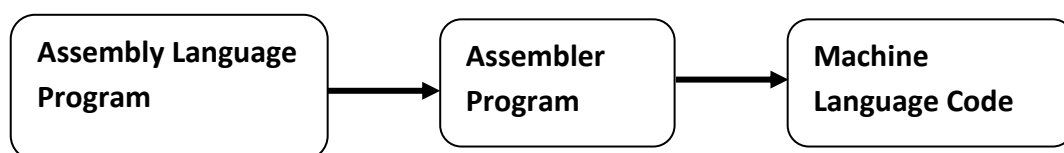
Microprocessor understands the language of 0's and 1's only .This language is called Machine Language

Assembly language it uses English like words to convey the action/meaning called as MNEMONICS

for example MOV-MOVE; ADD-Addition, SUB-Subtract etc

*Assembly language is specific to a given processor

- ❖ Microprocessor cannot understand a program written in Assembly language
- ❖ A program known as Assembler is used to convert a Assembly language program to machine language



The "assembly code" for a computer is a textual representation of the machine code giving the individual instructions to the underlying machine

Assembly language programmes are usually written in a standard form so that they can be easily translated to machine language by an assembler program (may be self on cross assembler). In general, the assembly language statements have four sectors in general known as fields

[Label:]	[Mnemonic]	[Operand	[;Comment]
----------	-------------	-----------	------------

Label field: The first field of ALP statement is the label field. A label is a symbol used to represent an address that is not specifically known or it is a name to be assigned to an instruction's location. The label field is usually ended with a colon (:).

2) Mnemonic field: This field contains the mnemonic for the instruction to be performed. Sometimes mnemonics are referred to as operation codes or opcode. It may consist of pseudo mnemonics.

3) Operand field: Operand field consists of operand and operands - either constants or variables with reference to the instruction in the mnemonic field. It may be any register, data, or address on which the instruction is to be performed.

4) Comment field: A very important part of an ALP is the comment field. For most assemblers, the comment field is started with a semicolon (;).

A simple Program

```
START:   IN    00H        ; Read Input Switches
          OUT  01H        ; turn on device according to switch position
```

JMP START ; go back to beginning and read the switches again

An instruction is usually organized as below

- ❖ Instruction = Opcode + Operand
- ❖ *Opcode* is binary code that is used by the processor to understand the instruction.
- ❖ *Operand* is the data item upon which processor has to perform operation, as dictated by the *Opcode*.
- ❖ The *Operand* can be specified in a number of ways.
- ❖ *Data Addressing Mode* is actually the method by which an *Operand* is specified in the instruction.

Data Addressing modes in 8085 microprocessor

An instruction may access data in variety of methods, called Data Addressing Modes. Following are the supported addressing modes in 8085.

- ❖ Register Addressing.
- ❖ Immediate Addressing.
- ❖ Direct Addressing.
- ❖ Indirect Addressing.
- ❖ Implicit Addressing

Register Addressing

The designated data item is present in one of the general purpose register of 8085. This mode is primarily used to specify *variables*.

e.g. MOV C, H

- + Copy the content of register H into register C.
- + The data item to be copied is present in a CPU register H.
- + Hence, it is *Register Addressing Mode*.
- + Execution of MOV C, H is explained next.

MOV R_D, R_S – Copy the content of Source Register (R_S) into Destination Register

(R_D).

- + R_S and R_D can be any of the general purpose registers (A, B, C, D, E, H, L).
- + Examples MOV A, D; MOV H, C; MOV A, A etc.
- + 49 such Opcodes are there for MOV R_D, R_S.
- + This is a 1-byte instruction.
- + Takes 1-Machine Cycle (Opcode Fetch) to execute.
- + None of the Flags are affected.

Example: MOV R_D, R_S , ADD R, SUB R, ADC R, SBB R etc

A Simple program:

Register A has data 20H and Register D has data 30H. Write an 8085 ALP (Assembly Language Program) that copies the data in A to Register E and data in D to Register L.

MOV E, A ; Copy Register A into Register E.

MOV L, D ; Copy Register D into Register L.

HLT ; Stop

Immediate Addressing

The designated data item *immediately follows* the Opcode and hence the name Immediate Addressing. This mode is used to specify the *constant data*.

e.g. **MVI C, 12H**

- + Copy the 8-bit data 12H into register C.
- + The data item (12H) to be copied is present immediately after the Opcode MVI C.
- + Hence, it is *Immediate Addressing Mode*.
- + Execution of MVI C is explained next.
- + Example: MVI R, DATA8;

A Simple program

Write an 8085 ALP (Assembly Language Program) that adds two data items 20H and 30H and store the result in Register C and E.

```

MVI A, 20H      ; Load 20H into Accumulator.
MVI B, 30H      ; Load 30H into B.
ADD B           ; Add the data items and store result in A.
MOV C, A        ; Get result in C.
MOV E, A        ; Get result in E.
HLT             ; Stop.

```

Example: MVI DATA8 ;ADI DATA8; ACI DATA8; SUI DATA8; SBI DATA8; LXI Rp

Direct Addressing

In this mode, the designated data item is stored in memory and the memory address is specified *directly* in the instruction.

e.g. **LDA 5000H**, *Load accumulator directly* from the contents of memory address 5000H.

$A \leftarrow (5000H).$

e.g. **STA 5000H**, *Store accumulator directly* at memory address 5000H.

$A \rightarrow (5000H).$

A Simple program

Two data items are stored at locations 2050H and 2051H. Write an 8085 ALP (Assembly Language Program) to copy these data at memory locations 2100H and 2101H.

```

LDA 2050H      ; Get data from 2050H in A.
STA 2100H      ; Store A at 2100.
LDA 2051H      ; Get data from 2051H in A.
STA 2101H      ; Store A at 2101.
HLT

```

Write an 8085 ALP to add two data items and store the result in memory.

MVI A, 20H; Load 1st Data in Accumulator.
MVI B, 30H; Load 2nd Data in B.
ADD B ; Add the data.
STA 2052H ; Store result in memory at 2052H.
HLT ; Stop.

LHLD and SHLD

- **LHLD 5000H, Load HL Pair *directly* from memory address 5000H.**

+ $L \leftarrow (5000H).$

+ $H \leftarrow (5001H).$

- ❖ **SHLD 5000H, Store HL Pair *directly* at memory address 5000H.**

+ $L \rightarrow (5000H).$

+ $H \rightarrow (5001H).$

Indirect Addressing

The designated data item is stored in memory and the memory address is specified through a register pair (i.e. *indirectly*) in the instruction. This addressing mode is most suitable for tabular processing.

e.g. **MOV B, M**

+ Copy an 8-bit memory data *indirectly* into register B, where memory address is specified by HL Pair.

+ $B \leftarrow ((HL)).$

+ Since memory address is not the part of instruction it is *indirect addressing*.

MOV R, M

+ Copy an 8-bit memory data *indirectly* into register R.

+ $R \leftarrow ((HL))$.

MOV M, R

- + Copy an 8-bit data *indirectly* into memory from register R.
- + $R \rightarrow ((HL))$.
- + R can be any of the 8-bit general purpose registers (A, B, C, D, E, H, L).
- + 14 Opcodes are there for both the MOVes.
- + They are 1-byte instructions but take 2-Machine Cycles to execute.
- + Execution of **MOV M, B** is explained next.

A Simple Program

Two data items are stored at locations 2500H and 2501H. Write an 8085 ALP to add two data items stored in memory and store the result in memory at 2502H.

```
LXI H, 2500H    ; Pointer to 1st Data in HL Pair.
MOV A, M        ; Get 1st Data in A.
LXI H, 2501H    ; Pointer to 2nd Data in HL Pair.
MOV B, M        ; Get 2nd Data in B.
ADD B           ; Add data.
LXI H, 2502H    ; Pointer to Result in HL Pair.
MOV M, A        ; Store result in memory at 2502H.
HLT             ; Stop
```

Example: INX Rp ; DCX Rp; ADD M; ADC M; SUB M; SBB M; MVI M, DATA8; LDAX Rp; STAX Rp

Implicit Addressing

The designated data item is stored in CPU registers or machine component but the name of operand is not defined in the instruction.

e.g. **XCHG**, *Exchange* the content of HL Pair with DE Pair.

+ $HL \leftrightarrow DE$.

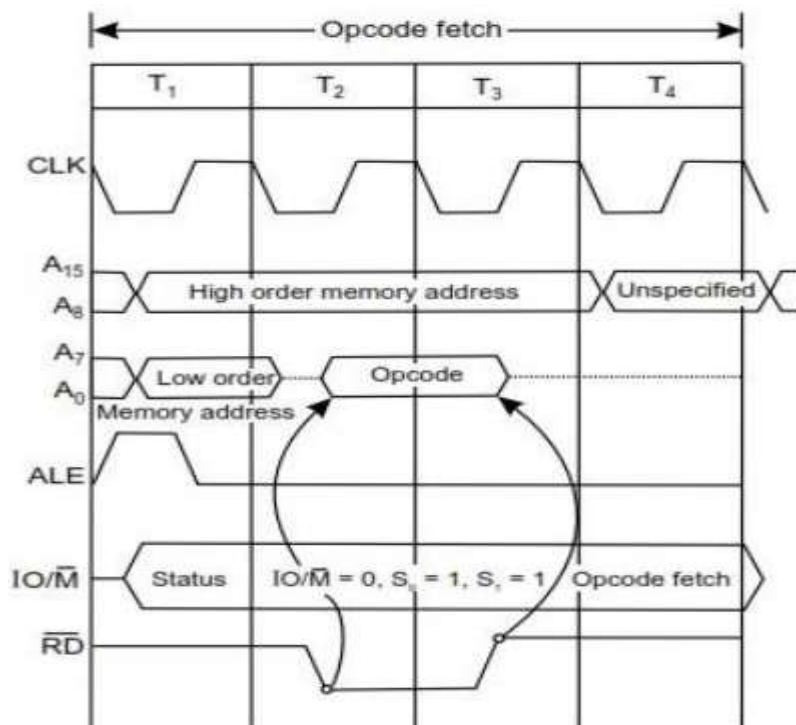
+ Since operand names HL or DE is not specified in instruction it is *Implicit Addressing*.

Timing diagram of 8085 microprocessor

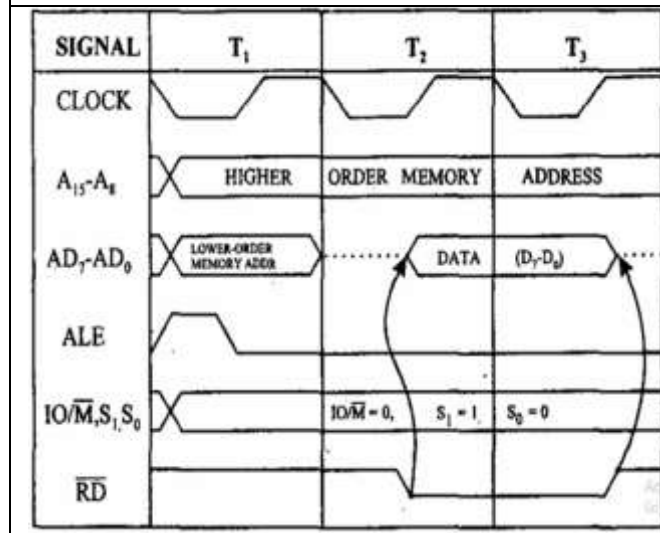
The 8085 microprocessor has 5 basic machine cycles. They are given as below

- ❖ Opcode fetch cycle (4T)
- ❖ Memory read cycle (3 T)
- ❖ Memory write cycle (3 T)
- ❖ I/O read cycle (3 T)
- ❖ I/O write cycle (3 T)

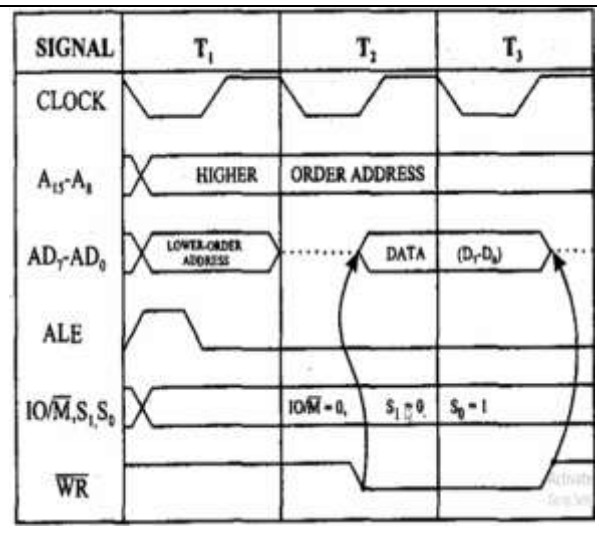
Opcode Fetch Machine Cycle



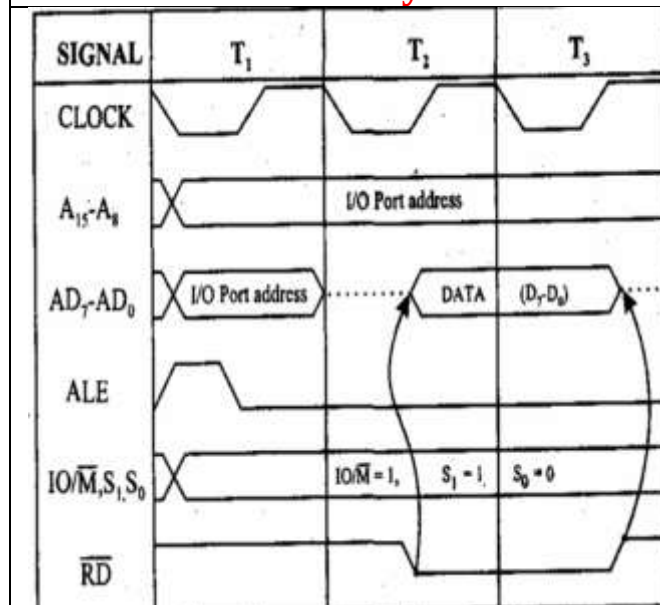
Memory Read Machine Cycle



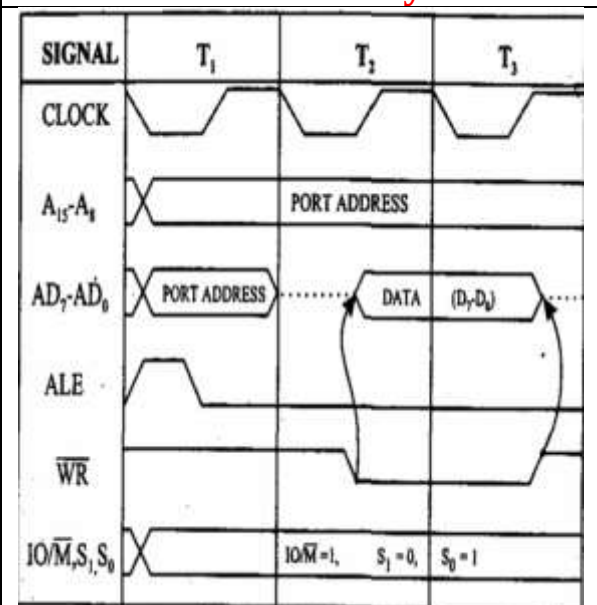
Memory Write Machine Cycle



I/O Read Machine Cycle



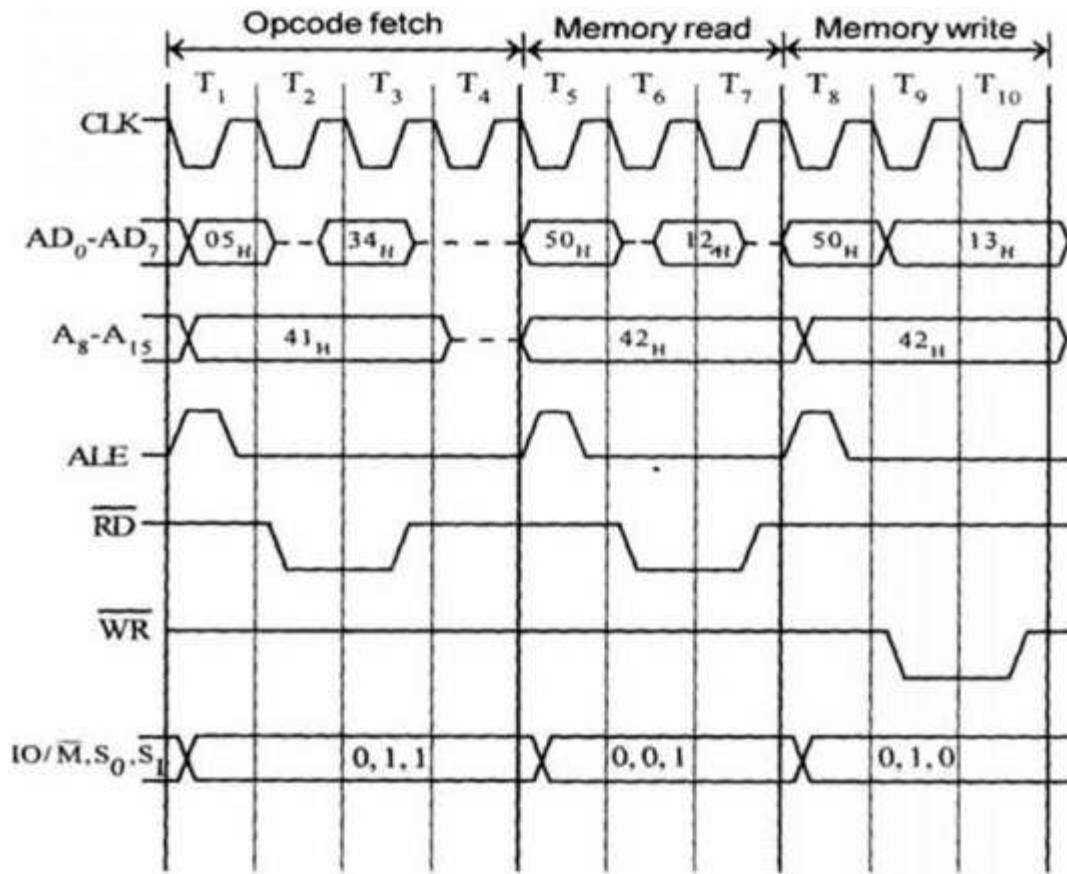
I/O Write Machine cycle



Timing diagram of INR M

Fetching the Opcode 34H from the memory 4105H. (OF cycle)

- ❖ Let the memory address (M) be 4250H. (MR cycle -To read Memory address and data)
- ❖ Let the content of that memory is 12H.
- ❖ Increment the memory content from 12H to 13H. (MW machine cycle)



Stack and Subroutine:

STACK:

- The stack in an 8085 microprocessor is a set of memory locations in the R/W memory.
- These memory location are used to store binary information(bytes) temporarily during the execution of the program.
- Stack is defined in the program LXI SP, which loads the 16-bit memory address in the stack pointer register.

- Stack is initialized at the highest available memory locations.
- Programmer and microprocessor shares the stack.
- Programmer can store and retrieve the contents of a register pair using PUSH and POP instructions.
- The microprocessor automatically stores the contents of program counter when a subroutine is called.
-

LXI	SP, 16 bit	-Load stack pointer -load stack pointer register with 16 bit address
PUSH	Reg. pair	-Push register pair onto stack -1 byte instruction
PUSH	B	-The contents of register pair are copied onto stack. -SP is decremented and the contents of high-order registers (B, D, H, A) are copied into stack. -SP is again decremented and the contents of low-order registers (C, E, L, Flags) are copied into stack.
PUSH	D	
PUSH	H	
PUSH	SP	
POP	Reg. pair	Pop stack to register pair
POP	B	-The contents of top of stack are copied into register pair. -The contents of location pointed out by SP are copied to the low-order register (C, E, L, Flags). -SP is incremented and the contents of location are copied to the high-order register (B, D, H,
POP	D	
POP	H	

POP	SP	A).
-----	----	-----

- **PUSH R_p** **R_p may be BC, DE or HL**

– E.g. PUSH D

$(SP-1) \leftarrow D$

$(SP-2) \leftarrow E$

$SP \leftarrow SP - 2$

- **PUSH PSW**

$(SP-1) \leftarrow A$

$(SP-2) \leftarrow FLAG$

$SP \leftarrow SP - 2$

– PUSH takes 3 machine cycles & 12 T-States for execution.

- **POP R_p** **R_p may be BC, DE or HL**

– E.g. POP B

$C \leftarrow (SP)$

$B \leftarrow (SP+1)$

$SP \leftarrow (SP + 2)$

- **POP PSW**

$FLAG \leftarrow (SP)$

$A \leftarrow (SP+1)$

$SP \leftarrow SP + 2$

– POP takes 3 machine cycles & 10 T-States for execution.

Example of PUSH and POP

1 0 0 1 0 1 0 0 = 94H

MVI B, 00H

MVI C, 94H

PUSH B

POP PSW

SUBROUTINE

- It is a group of instructions, which are written separately from the main program
- Subroutines are called many times in main program

E.g. Delay programs

- CALL(call a subroutine) and RET(return to main program from a subroutine) are two main instructions in 8085 to implement the subroutines.
- CALL instruction is used in main program to call a subroutine
- RET instruction is used at the end of the subroutine to return to main program

CALL 16-bit memory address of a subroutine

; call subroutine unconditionally

- 3-byte instruction that transfers the program sequence to a subroutine address
- Save the contents of the program counter on the stack
- Decrements the stack pointer register by two
- Jump unconditionally to the memory location specified by the second and third bytes. The second byte specifies a line number and third bytes specifies the page number
- This instruction is followed by a return instruction in the subroutine

8085 Registers and code:

Code	Registers	Code	Register Pairs
000	B	00	BC
001	C	01	DE
010	D	10	HL
011	E	11	AF or SP
100	H		
101	L		
111	A		
110	Reserved for Memory		

How to calculate the Opcode?

Ex. Add the contents of register to the accumulator

Opcode	Operands	Hex code
ADD	B	80H

5-bit Opcode	3 bits are reserved for a register
10000	SSS

ADD	10000
Reg. B to A	000
Binary Instruction:	10000 000= 80H

MOVE(Copy) the contents of Rs(Source) to Rd(destination)

2-bit Opcode for MOVE	3 bits for destination reg			3 bit for source register		
01	D	D	D	S	S	S

MOV B, A

2-bit Opcode for MOVE		3 bits for destination reg			3 bit for source register		
0	1	D	D	D	S	S	S
0	1	0	0	0	1	1	1

Opcode	Operands	Hex code
MOV	B, A	47H

MOV B, A

2-bit Opcode for MOVE		3 bits for destination reg.			3 bit for source register		
0	1	D	D	D	S	S	S
0	1	0	0	1	1	1	1

Opcode	Operands	Hex code
MOV	C, A	4FH